Engineering Applications of Artificial Intelligence 160 (2025) 111890

Contents lists available at ScienceDirect

Engineering Applications of Artificial Intelligence

journal homepage: www.elsevier.com/locate/engappai

Research on mobile robot path planning based on improved Q-evaluation ant colony optimization algorithm

Dongdong Li [a], Lei Wang [b,*]

[a] School of Mechanical and Electrical Engineering, Nanjing University of Aeronautics and Astronautics, Nanjing, 210000, Jiangsu, China
[b] School of Mechanical and Automotive Engineering, Anhui Polytechnic University, Wuhu, 241000, Anhui, China

ARTICLE INFO

Keywords:
Robot path planning
Q-evaluation
Ant colony optimization
Hyperparameter
Stability

ABSTRACT

This paper proposes an improved Q-evaluation ant colony optimization (IQACO) algorithm to address the shortcomings of traditional ant colony optimization (ACO) algorithm in solving path planning problems, such as the difficulty of effectively reflecting the quality of nodes through the concentration of pheromones, and the problem of poor search stability caused by excessive hyperparameters at different map sizes. Firstly, by analyzing the optimization process of the Q-Learning algorithm, the feasibility of evaluating nodes with Q-value was demonstrated. Secondly, an update method from path transformation to node Q-value was studied, and its superiority over pheromone concentration was demonstrated. Finally, a node selection strategy based on Q-evaluation was proposed to reduce the hyperparameters of the algorithm and improve its stability in optimizing environments of different sizes. To validate the effectiveness of the proposed improvement strategy, both theoretical analysis and repetitive experimental results demonstrate a significant reduction in the time complexity of the proposed algorithm compared to the traditional Ant Colony Optimization (ACO) algorithm. Additionally, simulations conducted on four different map sizes show that the reduction in hyperparameters enhances the algorithm's stability across various map scales. In addition, comparative experiments with other Q-learning-enhanced ACO variants further highlight the advantages of the proposed IQACO algorithm in terms of solution quality, convergence stability, and real-time performance. Furthermore, through comparisons with related work and recent studies, the proposed algorithm's superiority in terms of search capability and stability is clearly demonstrated.

1. Introduction

The proliferation of artificial intelligence technology has led to a significant expansion in the deployment of mobile robots across diverse sectors. The proliferation of mobile robots in intelligent manufacturing, logistics transportation, and workshop scheduling has significantly enhanced work efficiency and automation levels (Gui et al., 2023; J. Wu et al., 2021). Among the relevant technologies for mobile robots, path planning technology stands out as one of the key technologies, directly impacting the robot's autonomous navigation efficiency and task execution capabilities. Improving the path planning ability of mobile robots not only helps to enhance their intelligence, but also enhances their work efficiency in complex environments (Wang, 2020). Thus, conducting thorough research on mobile robot path planning is essential, as it plays a crucial role in advancing robotic technology and broadening its range of applications (Gui et al., 2023; Han et al., 2023).

At present, there are various types of intelligent algorithms in the field of path planning, mainly including genetic algorithm (Bo et al., 2024; Ning et al., 2023; Xu et al., 2023), particle swarm algorithm (S. Li et al., 2023; Lin et al., 2023; Sun and Yang, 2023), ant colony optimization algorithm (Cui et al., 2024; Liang et al., 2023; Y. Ni et al., 2023), artificial potential field algorithm (M. Li et al., 2023; Rao et al., 2023), and Q-Learning algorithm (Hao et al., 2023; Wang et al., 2024; Zhou et al., 2024). Among these, ant colony optimization (ACO) algorithm can effectively solve complex optimization problems by simulating ant foraging behavior. However, despite its numerous advantages, the traditional ACO algorithm suffers from limitations such as slow convergence rates and the tendency to get trapped in local optima. Addressing these issues is crucial to improving the performance of path planning in mobile robots.

This paper proposes an improved Q-evaluation ant colony optimization (IQACO) algorithm to solve path planning problems. The key

* Corresponding author.
E-mail addresses: lddya1996@nuaa.edu.cn (D. Li), wllx2010@ahpu.edu.cn (L. Wang).

https://doi.org/10.1016/j.engappai.2025.111890
Received 1 September 2024; Received in revised form 1 July 2025; Accepted 25 July 2025
Available online 5 August 2025

contributions of this work are as follows:

(1) Q-value based evaluation mechanism: In order to find a method that better reflects the advantages and disadvantages of path nodes, inspired by the Q-Learning algorithm, this paper introduces the use of Q-values to evaluate different nodes. It also proposes a method to map the advantages and disadvantages of mobility schemes to the advantages and disadvantages of nodes.
(2) Improved node selection strategy: To enhance the optimization ability of the algorithm through the use of node Q-values, this paper proposes a novel Q-based node selection strategy, replacing the traditional node selection strategy in the ACO algorithm.
(3) Stability enhancement across environments: By integrating the aforementioned strategies into the traditional ACO algorithm, the number of hyperparameters is reduced. Experimental results demonstrate that the proposed algorithm exhibits better stability and faster convergence in environments of various sizes.

The rest of this study is organized as follows: Section 2 reviews the related work. Section 3 provides detailed descriptions of environmental modeling and robot movement rules. Section 4 introduces the two strategies proposed within the IQACO algorithm. Section 5 presents the experiments conducted and analyzes the results. Section 6 discusses the managerial-level impact of the proposed method in practical manufacturing scenarios. Section 7 concludes the paper and outlines directions for future research.

2. Related work

2.1. Traditional ACO for path planning

ACO is widely used in various optimization problems, as it mimics the behavior of real ants searching for food. The algorithm's high adaptability and strong global search capabilities have led to its success in many fields (Zhang et al., 2021). Several studies have explored different improvements and applications of ACO algorithm. Ni et al. (Z. Ni et al., 2023) proposed a method to address the issue of energy efficiency in routing algorithms by introducing Power-Saving Ant Colony Optimization. This approach considers the residual energy of wireless sensors as a parameter, aiming to efficiently and swiftly determine the optimal route between the source and destination nodes. To tackle the problem of low pickup efficiency in robotic arm harvesting of safflower filaments, Guo et al. (2024) introduced a trajectory planning approach utilizing an ant colony genetic algorithm. This approach features enhancements via an adaptive adjustment mechanism, pheromone constraints, and the inclusion of optimized parameters derived from genetic algorithms. Zhang et al. (2024) introduced the Mediocrity Ant Colony Algorithm (MACA) for performing edge detection on colony images. MACA incorporates the mediocrity rule along with empirical functions to construct a pheromone database. This database acts as a reference for updating pheromones. The algorithm utilizes the Chebyshev distance as a weighting factor to influence these updates. Additionally, it integrates heuristic information acquisition using the maximum variance classification method and local path weights. Wang et al. (Wang and Han, 2021) introduced a novel hybrid algorithm, combining symbiotic organisms search with ACO as SOS-ACO, aimed at tackling the TSP problem. In contrast to conventional ACO that necessitate manual assignment of all parameters, SOS-ACO simplifies this process by requiring initialization of only certain parameters, with the remainder being adaptively optimized through SOS algorithm. Yang et al. (2021) introduced an enhanced ant colony optimization (EACO) algorithm designed to manipulate the scattered light field resulting from an incident beam traversing a turbid medium. EACO modifies the search and pheromone deposition rules of the conventional ACO, thereby enhancing its search capabilities. By strategically defining the cost function, EACO algorithm effectively molds the scattered light field through turbid media into a predefined configuration. Gao et al. (2022) introduce a novel hybrid ACO approach for addressing the Capacitated Vehicle Routing Problem (CVRP), focusing on minimizing the distances traveled by vehicles. The proposed method integrates the Fireworks Algorithm (FWA) into the ACO framework to enhance algorithm diversity and prevent premature convergence to local optima. Moreover, the algorithm integrates an elite ant colony strategy to enhance the attractiveness of locally optimal paths in the exploration phase, alongside the Max-Min Ant System (MMAS) which regulates pheromone levels to prevent excessive accumulation on particular routes.

2.2. Improvements to traditional ACO

The Ant Colony Optimization (ACO) algorithm, while effective in solving complex path planning problems, faces several challenges, such as slow convergence rates, susceptibility to local optima, and issues related to the balance between exploration and exploitation. To address these limitations, researchers have introduced various enhancements to the traditional ACO algorithm (Tan et al., 2022; Yang et al., 2019). These improvements can be categorized into three main areas: hybridization with other algorithms, improvement in convergence speed and exploration capabilities, and enhancement of search efficiency and path quality.

2.2.1. Hybridization with other algorithms

One of the most common approaches to improve ACO is hybridizing it with other optimization algorithms. This hybridization aims to leverage the strengths of both algorithms, combining the global search capability of ACO with the specific advantages of other methods.

In addressing the challenges related to the path planning of intelligent vehicles, Shi et al. (2022) developed an advanced hybrid algorithm combining GA and ACO algorithm, along with the creation of a physical vehicle model. The study first enhanced the ACO algorithm by incorporating an adaptive evaporation factor into the heuristic function. Following this, improvements to the GA were made by optimizing the fitness function and adaptively adjusting the crossover and mutation rates. Ultimately, the hybrid algorithm was further refined by adding a deletion operator, employing an elite retention strategy, and incorporating suboptimal solutions from the improved ACO algorithm into the enhanced GA to produce optimized new populations. Xiang et al. (2025) proposed a hybrid approach called DIACO, which integrates the Dijkstra algorithm with ACO for path planning in nuclear radiation environments. Their method incorporates the shortest path generated by Dijkstra to influence the initial pheromone distribution, enhances the heuristic function, and adopts the Max-Min Ant System framework to improve convergence speed. This combination not only reduces the cumulative radiation dose but also leads to shorter and smoother paths, demonstrating the effectiveness of multi-strategy fusion under dynamic and hazardous conditions. Tan et al. (2024) addressed the Electric Vehicle Charging Route Planning problem by introducing an Improved Ant Colony Optimization (IACO) algorithm tailored to user-specific constraints. Their method combines heuristic refinement with a topology-based optimization framework to reduce computational overhead. Furthermore, they integrated a discrete electricity dynamic programming (DE-DP) approach to jointly optimize path selection and charging schedules. This hybrid framework demonstrated superior performance in balancing route efficiency and charging requirements, highlighting the potential of ACO variants in multi-constraint environments.

Despite these advancements, hybrid algorithms can introduce additional complexity and computational overhead, making them challenging to apply in real-time systems.

2.2.2. Improving convergence speed and exploration capabilities

To tackle the issue of slow convergence, several researchers have focused on modifying the pheromone update and search strategies.

To tackle the issues of slow convergence speed, inefficiency, and the propensity to fall into local optima in traditional ACO, Wu et al. (2023) propose a modified adaptive ACO algorithm (MAACO). In MAACO, a novel heuristic mechanism incorporating orientation information is introduced to guide the direction during iterations, thereby accelerating the algorithm's convergence speed. Additionally, an enhanced heuristic function is designed to improve goal-oriented behavior and minimize the number of turns in the planned path. Furthermore, an improved state transition probability rule is implemented to substantially boost search efficiency and increase swarm diversity. Lastly, a new approach for unevenly distributing the initial pheromone concentration is applied to prevent blind searching. Miao et al. (2021) introduced an Enhanced Adaptive ACO (EACO) algorithm aimed at resolving the challenges of extended path planning times, slow convergence rates, and susceptibility to local optima in indoor mobile robots employing traditional ACO. The EACO algorithm enhances the ACO transition probability by integrating an angle guidance factor and an obstacle exclusion factor, thereby improving both real-time performance and safety in path planning. Additionally, it features a heuristic information adaptive adjustment factor and an adaptive pheromone evaporation factor within the ACO pheromone update mechanism to strike a balance between convergence speed and global search effectiveness. To devise an effective and dependable global path for unmanned surface vehicles (USVs), Cui et al. (2022) developed a global path planning algorithm for USVs that leverages an optimized ant colony algorithm (OACA). To achieve the global optimal solution of the model and expedite the convergence rate, the initialization phase of the ACO algorithm incorporates an uneven distribution of initial pheromone, influenced by the distance relationships between the intermediate node, the starting point, and the endpoint, thereby enhancing the initial search efficiency. During the iterative search for the optimal solution, the algorithm introduces a weight factor to augment the influence of the current optimal solution on subsequent ants, refining the pheromone update rule to accelerate the convergence of the algorithm. Li et al. (2025) proposed an Intelligently Enhanced Ant Colony Optimization (IEACO) algorithm that incorporates a series of mechanisms to boost the global search capabilities of ACO. Specifically, the algorithm introduces a non-uniform initial pheromone distribution to accelerate early-stage exploration, and applies an ε-greedy strategy to better manage the trade-off between exploration and exploitation. Moreover, the exponents controlling pheromone influence (α) and heuristic factors (β) are adaptively adjusted during the search process, allowing for dynamic balancing as the environment changes. To further improve path quality, a multi-objective heuristic function combining target distance and turning angle is employed to refine node selection. The algorithm also features a dynamic global pheromone update mechanism that helps avoid premature convergence. In addition to the above studies, Sun et al. (2025) introduced the Emergency Path-planning Improved Ant Colony Optimization (EPIACO) algorithm to address stagnation and convergence latency issues that frequently occur in traditional ACO approaches. EPIACO integrates multiple enhancements, including a non-uniform initialization of pheromones, a direction-sensitive heuristic function, and an adaptive evaporation rate mechanism. Moreover, the algorithm adopts a refined pheromone update scheme and an adjusted transition probability model, contributing to both accelerated convergence and improved path smoothness.

However, in large-scale environments with multiple dynamic obstacles, such approaches may still struggle to maintain the global search capabilities required for optimal solutions.

2.2.3. Enhancing search efficiency and path quality

Improving search efficiency and ensuring high-quality paths are also critical aspects of ACO improvements. Researchers have focused on refining the node selection process and introducing adaptive strategies to optimize path quality.

To overcome the limitations of the Ant Colony Optimization in mobile robot path planning, Liu et al. (Liu and Liu, 2023) introduced an improved ant colony algorithm that includes rapid optimization and a hybrid prediction mechanism. This algorithm incorporates a composite optimal node prediction model into the state transition function to increase the chances of selecting the best nodes during path planning. Additionally, it employs a pheromone model with an initial distribution and a "reward or punishment" update mechanism to strategically adjust the global pheromone levels. This approach boosts the pheromone concentrations on superior path nodes, thereby enhancing the heuristic guidance. Chen et al. (2022) introduce the JPIACO algorithm, which enhances the pathfinding accuracy of the traditional ACO algorithm. The JPIACO algorithm initially utilizes an initial pheromone concentration distribution derived from jump points to guide the pathfinding process, thereby accelerating early iterations. Additionally, it incorporates a turning cost factor into the heuristic function to improve path smoothness. The algorithm also integrates adaptive reward and punishment factors along with the Max-Min ant system to boost both the iteration speed and the algorithm's global search capability. Simulation results on three different-sized maps indicate that the JPIACO algorithm significantly enhances pathfinding accuracy and minimizes the number of turns. Chen et al. (2025) introduced a variable-scale Improved Ant Colony Optimization (IACO) algorithm tailored for land leveling path planning. The proposed method includes two variations: a large-scale inter-regional planning algorithm (IACO) and a fine-grained full-field version (FIA*ACO), which is a fusion of Improved A* and ACO. By dynamically adjusting the planning scale, the algorithm significantly improved the flatness and leveling precision across different terrain scenarios. Field experiments confirmed that this variable-scale strategy effectively enhanced both the global distribution and local refinement of elevation differences, demonstrating the algorithm's capacity to deliver high-quality paths in practical agricultural applications.

These methods have shown promising results in terms of both search efficiency and path quality, but they often require fine-tuning and additional computational resources to adapt to various problem sizes and environments.

2.3. Research gap

Although the aforementioned studies have significantly advanced the performance of the Ant Colony Optimization algorithm, several critical challenges remain unresolved. For instance, the pheromone update mechanism plays a central role in summarizing the search outcomes of the ant population. However, non-optimal paths often contain redundant information that can mislead the search process. Traditional ACO algorithms address this issue through pheromone evaporation, which gradually eliminates the influence of poor solutions. Nevertheless, this process typically requires many iterations, thereby slowing down convergence. Consequently, how to utilize search results more efficiently to guide the behavior of subsequent ants remains a key challenge.

In addition, ACO algorithms often lack generalization across different map sizes. Changes in the scale of the environment can significantly affect the values of the heuristic function, which in turn alters the influence of pheromone concentration in the node selection process. As a result, achieving good performance usually requires manual parameter tuning for different scenarios. This sensitivity to scale undermines the adaptability and stability of the algorithm.

To address these issues, this paper proposes an Improved Q-evaluation Ant Colony Optimization (IQACO) algorithm. Inspired by the concept of Q-values in Q-Learning, the IQACO algorithm introduces a Q-based evaluation mechanism to more accurately assess node quality. Furthermore, a novel node selection strategy is designed to better balance exploration and exploitation, thereby improving the stability and effectiveness of path planning across environments of varying sizes. The following sections will elaborate on the algorithm design and present experimental results to validate its effectiveness.

3. Problem model for path planning

In this section, the environmental modeling methods and robot movement rules involved in this paper are described.

3.1. Environmental modeling

In path planning research, the primary task is environmental modeling. To date, numerous environmental modeling methods have been applied to path planning challenges, mainly including grid models methods, topology graph methods, and visual graph methods (Tian et al., 2020). Among these, the grid method is particularly popular due to its simplicity and ease of implementation. Consequently, this paper selects the grid method for environmental modeling (Y. Wu et al., 2021). In grid models, blank grids are usually used to represent movable grids, and black grids are used to represent obstacle grids, that is, impassable grids (Huang et al., 2021). The schematic diagram of a grid environment model is shown in Fig. 1.

To ensure that the grid-based environment accurately reflects the spatial constraints in real-world scenarios, this work adopts an obstacle expansion strategy. Specifically, real-world obstacles are enlarged by a margin equal to the sum of the robot's radius and a predefined safety buffer. This expansion process allows the robot to be treated as a particle during planning, ensuring that potential collisions with the boundaries or corners of obstacles are avoided. The expanded obstacles are then mapped onto the grid, and the overlapped grid cells are marked as impassable (Shi et al., 2022). This process is illustrated in Fig. 2.

3.2. Robot movement rule

In a grid diagram, it is usually agreed that robots can only reach the adjacent 8 nodes (required to be passable nodes) during each movement (Chaari et al., 2017; Miao et al., 2021). If the current node coordinates of the robot are (x,y), the calculation of its adjacent node list *neighbors* is shown in Eq. (1). For example, when the robot is located at nodes [10, 10] in the environment shown in Fig. 1, the adjacent nodes (passable nodes) it can reach are [9,10], [9,9], and [10,9], respectively.

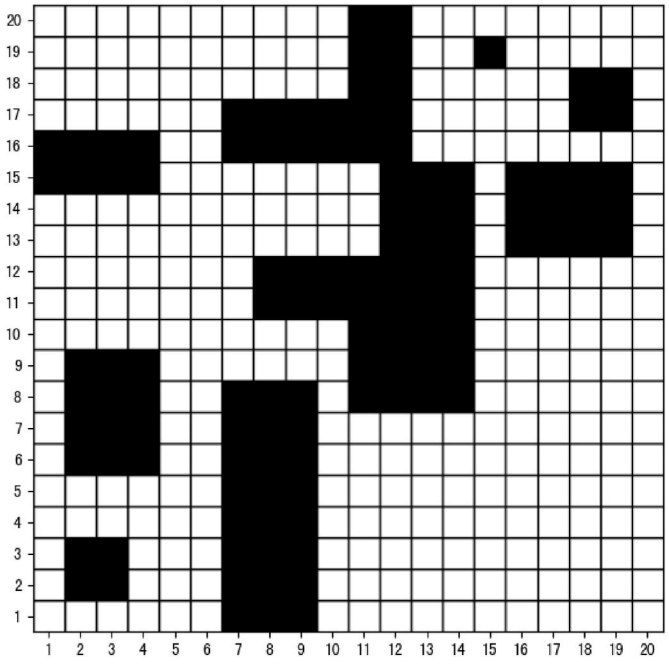


Fig. 1. Schematic diagram of grid environment model.

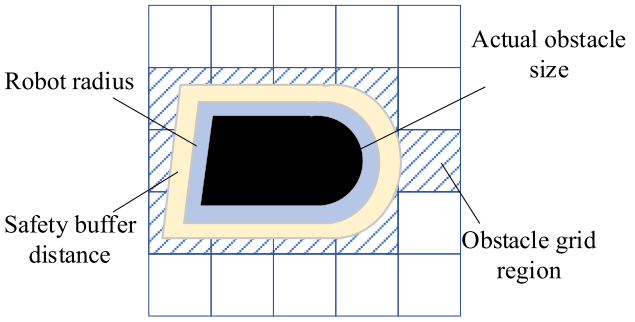


Fig. 2. Illustration of obstacle expansion before grid mapping.

$$neighbors = \{(x',y') | x' \in [x-1, x+1], y' \in [y-1, y+1], (x',y') \neq (x,y)\} \tag{1}$$

4. IQACO algorithm

In this section, the Q-evaluation strategy and a node selection strategy based on Q-evaluation are proposed to improve the performance of traditional ant colony optimization (ACO) algorithm, and then an improved Q-evaluation ant colony optimization (IQACO) algorithm is proposed. By analyzing the effectiveness of using Q-value to evaluate nodes, and proposing a method for calculating the Q-value of nodes based on the path obtained by ants, and designing a node selection strategy based on Q-evaluation, the optimization ability and stability of the algorithm have been improved. Finally, the main content of IQACO algorithm was disclosed in the form of a flowchart and pseudocode. The complete source code implementing IQACO is available at https://github.com/***/IQACO, published after the paper.

4.1. Q-evaluation strategy

4.1.1. The effectiveness of Q-evaluation

In ACO, the method for selecting nodes on the optimal path mainly relies on the mechanism of pheromone concentration accumulation and evaporation. However, this mechanism makes it difficult to select good nodes in the early iterations of the algorithm. Eq. (2) is the calculation method for the pheromone concentration of a node, where $\Delta\mu_{ij}^{n}$ is the pheromone concentration left by the n-th ant moving from node i to node j, Ph is the total amount of pheromone left by one ant, L_n is the total length of the path generated by the ant, and L_{ij} is the Euclidean distance between node i and node j. Analyzing Eq. (2), it can be seen that the pheromone concentration of a node mainly depends on the overall evaluation value of the path it is on and the total amount of pheromone that a single ant can produce. This amount is usually constant, and in the early iterations, the resulting paths often contain redundant segments. Therefore, the overall quality of the path cannot reflect the quality of individual nodes, which seriously interferes with the efficiency of selecting superior nodes in the path.

$$\Delta\mu_{ij}^{n} = \frac{Ph \cdot L_{ij}}{L_n^2} \tag{2}$$

The Q-Learning algorithm, a classic algorithm in reinforcement learning, has numerous applications in the field of path planning. Considering that in Q-Learning, the Q-value corresponding to action a (move mode a) in state s (current node s) to some extent reflects the advantages and disadvantages of the next state s' (new node s' after movement). That is, the larger the Q-value, the greater the expected reward value corresponding to the next state C. Moreover, in Q-Learning algorithm, the evaluation unit for the solution is no longer a path, but accurate to the node, which provides a new approach for node selection in algorithm.

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right] \tag{3}$$

Eq. (3) is the update formula for the Q-value corresponding to taking action a in state s in Q-Learning, where α is the learning rate, r is the reward value for action a (typically set to 1 upon reaching the goal and 0 otherwise), γ is the discount factor, and $\max_a Q(s',a')$ is the maximum Q-value among all possible actions a' in the next state s'. Given the start point [1,1] and the end point [4,4] for path planning, the discount factors corresponding to straight and diagonal movements are set as $\gamma_l = 0.9$ and $\gamma_s = 0.85$, respectively. Considering that in Eq. (3), $\max_a Q(s',a')$ represents the expected reward value of state s', the mapping relationship between the Q-value $Q(s)$ of node s and $Q(s,a)$ in Q-Learning is established as shown in Eq. (4).

$$Q(s) \leftarrow \max_a Q(s,a) \tag{4}$$

Fig. 3 illustrates the $Q(s)$ values of each node after path planning training using the Q-learning algorithm on a 4x4 grid map, with the start point at [0,0] and the end point at [3,3]. From the data in Fig. 3, it is evident that the difference in Q-value between any two adjacent nodes depends entirely on the discount factor γ. For instance, the Q-value of node [4,4] and its adjacent nodes [3,4] and [3,3] are $Q_{a=[3,4]} = 0.9 = 1^* \gamma_l = 1^*0.9$ and $Q_{a=[3,3]} = 0.85 = 1^* \gamma_s = 1^*0.85$, respectively. Clearly, nodes farther from the endpoint exhibit greater Q-value decay. In the later iterations, Q-Learning employs a greedy strategy to generate paths, meaning it selects the adjacent node with the highest Q-value for each move. Nodes with the highest Q-value experience the least decay, indicating they are closer to the endpoint. Therefore, Q-Learning can find the optimal solution by searching nodes with minimal Q-value decay. In summary, evaluating node quality based on Q-value proves effective and feasible.

4.1.2. Update method of Q-value

In Q-Learning, the quality of nodes can only be evaluated using Q-value. Therefore, a learning rate is set to prevent premature convergence. However, in ACO, the distance heuristic function can evaluate node's quality, and the roulette wheel method also can provide the ability to escape local optimal solutions. Hence, in constructing the Q-evaluation strategy, this paper discards the use of a learning rate. As mentioned in Section 2.2.1, a node's Q-value can be regarded as the Q-value at the end point continuously decaying along the path. Thus, during the Q-value update process, it is only necessary to retain the maximum Q-value of the node, which corresponds to preserving the optimal path from the node to the end point. This will help in filtering out poor path information. Considering a path list $P = \{S...s,s',...E\}$ where S and E are the start and end points, respectively. The preliminary design for the Q-value update formula of a path node s is given by Eq. (5). Furthermore, by initializing the Q-value $Q(E)$ of the end point E to the reward value r, Eq. (5) can be further optimized to remove the reward value. Hence, the final Q-value update formula for node s in this paper is defined by Eq. (6). The update method of Q-value is illustrated in Algorithm 1.

$$Q(s) \leftarrow \max(r + \gamma \cdot Q(s'), Q(s)) \tag{5}$$

$$Q(s) \leftarrow \max(\gamma \cdot Q(s'), Q(s)) \tag{6}$$

Algorithm 1. Update Method of Q-value

Input: path node list ***P***, Q-value matrix of nodes Q-Matrix,
Output: Q-Matrix
For j **from Length**(P) - 2 **to** 0 **step** −1 **do**:
  If Distance(P[j], P[j+1]) = = 1 **then**://Distance *()* is a function for finding distance
    $\gamma = 0.9$;//Discount factor
  Else:
    $\gamma = 0.85$;
  End If
  Q-Matrix[*P*[j]] = **Max** (γ**Q-Matrix* [P [*j*+1]], Q-Matrix [P [j]]);
End For
Return Q-Matrix.

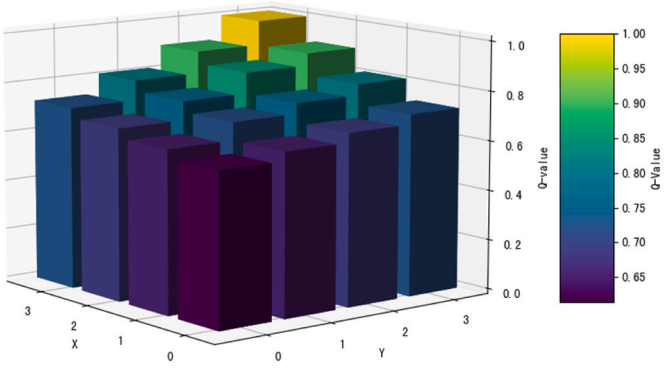
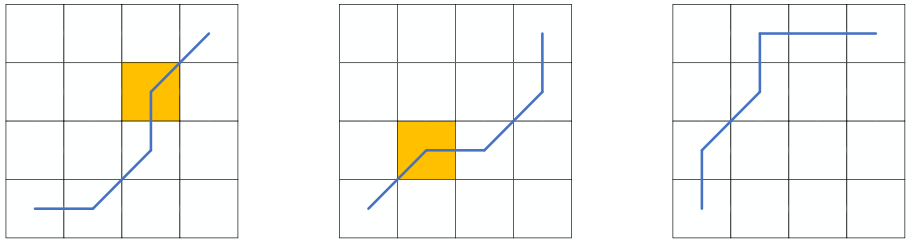


Fig. 3. 3D bar chart of Q values for each node.

To demonstrate the effectiveness of using Q-value to measure the quality of path nodes, three non-optimal paths were selected as test paths in the blank grid environment described above. Among them, Path 1 and Path 2 each include one highlighted node from the optimal path, while Path 3 does not include any nodes from the optimal path, as shown in Fig. 4. Based on the information from these three paths, the Q-value update process illustrated in Algorithm 1 is simulated and compared with the pheromone concentration update method in traditional ACO algorithm ($Ph = 1$). The results are shown in Fig. 5. From the results, the inability of pheromone concentration to effectively measure node quality is mainly due to poor nodes being selected multiple times, leading to excessively high pheromone concentrations, such as node [3,2] in Path 1 and Path 2. In contrast, the Q-value evaluation method can effectively measure the quality of nodes because a node's Q-value is only related to its quality and not to the number of times it is selected. This improves the efficiency of selecting high-quality nodes and reduces the optimization cost of the algorithm. For instance, even though node [3,2] is selected multiple times, the Q-value evaluation mechanism only retains the Q-value data of the known optimal path. Moreover, since the path evaluation is decomposed into node evaluations, it is possible to effectively select better nodes to move to during the offspring path generation, thereby achieving a combination of high-quality nodes and further improving the algorithm's performance. For example, in the optimal path, node [2,2] from Path 2 moves to node [3,3] from Path 1. The above content sufficiently demonstrates the feasibility and effectiveness of the Q-value update method proposed in this paper.

4.2. Node selection strategy based on Q-evaluation

In the traditional ACO algorithm for path planning, an ant's movement primarily relies on the distance heuristic function and the pheromone concentration heuristic function. The distance heuristic function is mainly influenced by the distance between the candidate node and the end point, while the pheromone concentration heuristic function is influenced by the pheromone concentration distributed on the nodes. Eq. (7) represents the node selection probability function in ACO, where $p_{ij}^n(t)$ is the probability of the n-th ant in the t-th generation moving from node i to node j, $\tau_{ij}(t)$ is the distance heuristic function from node i to node j, ω is the distance heuristic factor, $\eta_{ij}(t)$ is the pheromone concentration heuristic function, β is the pheromone heuristic factor, and $allowed_n$ is the list of feasible nodes.



(a). Path 1 (b). Path 2 (c). Path 3

Fig. 4. Three non-optimal paths.

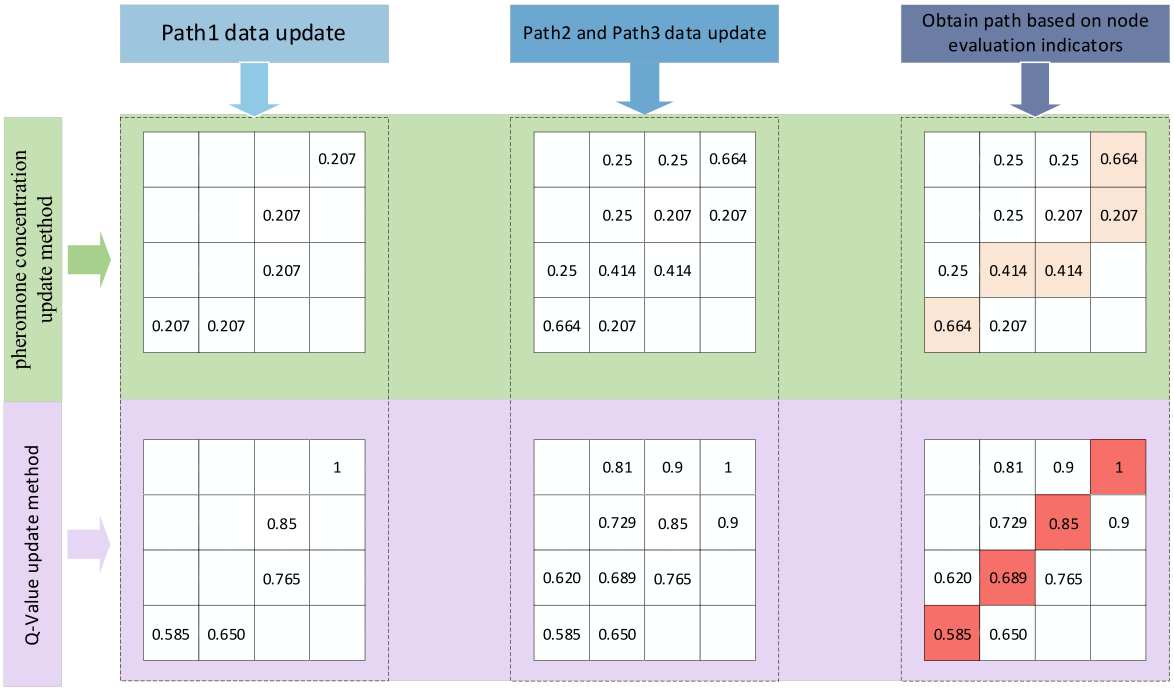


Fig. 5. Comparison of two updating methods.

$$p_{ij}^{n}(t)=\begin{cases}\dfrac{[\tau_{ij}(t)]^{\omega}\cdot[n_{ij}(t)]^{\beta}}{\sum_{s\in\ allowed}[\tau_{is}(t)]^{\omega}\cdot[\eta_{is}(t)]^{\beta}}, & s\ \varepsilon\ allowed_n\\ 0, & s\notin allowed_n\end{cases}\tag{7}$$

The traditional ACO algorithm has a drawback of requiring parameter adjustment for different map sizes. The main reason for this is that the operator between $\tau_{ij}(t)$ and $\eta_{ij}(t)$ in Eq. (7) is multiplication. Clearly, with different maps, the distance between the start and end point changes, which affects the value of $\tau_{ij}(t)$. Meanwhile, in the early iterations $\eta_{ij}(t)$ is mainly influenced by the initial pheromone concentration (usually a constant value). This requires ω and β to be adjusted appropriately to maintain the optimization efficiency of the algorithm. Based on the above analysis, this paper reconstructs the node selection probability formula using Q-value, discarding the use of pheromone concentration. To ensure the balance between the distance heuristic function and the node Q-value when constructing the selection probability, the operator between them is changed to addition. The new selection probability formula is designed as Eq. (8). Here, λ is the weight factor for the Q-value. To ensure population diversity, the calculation formula for λ is designed as shown in Eq. (9), where N is the population size. To further improve the utilization efficiency of Q-value, Furthermore, to improve the efficiency of Q-value utilization, a λ-greedy strategy selection method is designed, inspired by the ε-greedy strategy in Q-Learning algorithm, to balance the algorithm's exploration and exploitation capabilities. Finally, a node selection strategy based on Q-evaluation is proposed, as shown in Eq. (10).

$$p_{ij}^{n}(t)=\begin{cases}\dfrac{(1-\lambda)\cdot\tau_{ij}(t)+\lambda\cdot(1+Q(j))}{\sum_{s\epsilon\ allowed_n}((1-\lambda)\cdot\tau_{is}(t)+\lambda\cdot(1+Q(S)))}, & s\ \epsilon\ allowed_n\\ 0, & s\notin allowed_n\end{cases}\tag{8}$$

$$\lambda=\frac{n}{N}\tag{9}$$

$$\pi(i)=\begin{cases}\underset{s\in allowed_n}{\operatorname{argmax}}p_{is}^{n}(t), \text{if rand}()<\lambda\\ \text{Roulette Wheel Selection}, P=Eq.(6), else\end{cases}\tag{10}$$

To verify the effectiveness of the proposed strategy, the map shown in Fig. 1 was chosen as the simulation environment. The start and end point were set to [1,1] and [20,20], respectively. The population size was set to 30, with a maximum of 50 iterations, and the simulation was run 20 times continuously. The performance of the following three node selection strategies was compared, with the data presented in Table 1 and Fig. 6. Table 1 outlines the primary evaluation metrics, which include initial path quality, convergence accuracy, and the minimum number of iterations required. Initial path quality serves as a key indicator for evaluating node quality through Q-values. The convergence

Table 1
Simulation data of three selection strategies.

Strategy	Initial path quality		Convergence results		Minimum number of iterations	
	average	optimal	average	optimal	average	optimal
Strategy 1	60.3448	49.7989	36.6271	36.3847	31	20
Strategy 2	**38.3847**	**38.3847**	35.7989	35.7989	3	2
Strategy 3	43.5681	**38.3847**	**34.3847**	**34.3847**	5	3

outcomes demonstrate the stability and effectiveness of each method in addressing the problem. Meanwhile, the minimum number of iterations reflects the exploratory efficiency of each strategy. Fig. 6(a) illustrates the final paths produced by the three strategies, while subfigures (b)–(d) present the iteration curves for each, offering a clear visual representation of their optimization behavior and convergence performance. Fig. 7 depicts the relationship between the Q-value and path length under Strategy 3, providing further insight into its behavior.

- Strategy 1: Node selection method of traditional ACO algorithm. In this strategy, the mobile node will be selected by the roulette wheel method, and the probability of node selection will be calculated by Eq. (7).
- Strategy 2: Greedy selection method based on Q-value. In this strategy, the node with the maximum Q-value is selected.
- Strategy 3: Node selection strategy based on Q-Evaluation proposed in this paper.

From the data in Table 1, compared to Strategy 1, both Strategy 2 and Strategy 3 have significant advantages in initial path quality, which proves that replacing pheromone concentration with Q-value is feasible. Additionally, analyzing the simulation results of Strategy 2 reveals that solely using Q-value can quickly obtain a relatively good initial population, but it lacks sufficient exploration capability, causing the algorithm to quickly fall into local suboptimal solutions. In contrast, Strategy 3 demonstrates good ability to break out of local suboptimal solutions. While retaining the advantage of obtaining a better initial population through Q-value, Eq. (8) endows the algorithm with sufficient exploration capability to prevent it from falling into local suboptimal solutions. The average path length per generation shown in Fig. 6(d) also demonstrates the diversity of paths in each generation. Fig. 7 illustrates the relationship between the paths and ant IDs in a particular generation under Strategy 3, where the λ value for the first ant is 0 and gradually increases with the ant ID. It can be observed that when the λ value is small, the path length is longer, indicating minimal use of Q-value. As the λ value increases, the influence of Q-value on decision-making begins to grow, and the path length shows a decreasing trend. This proves that the methods for move strategy selection and Q-value utilization through Eq. (9) and Eq. (10) in this paper are effective.

4.3. Steps of IQACO

In this section, Integrate the above improvement strategies with traditional ACO algorithm and propose an improved Q-evaluation ant colony algorithm (IQACO). The pseudocode of IQACO is shown in Algorithm 2. Moreover, Fig. 8 gives the flow chart of IQACO.

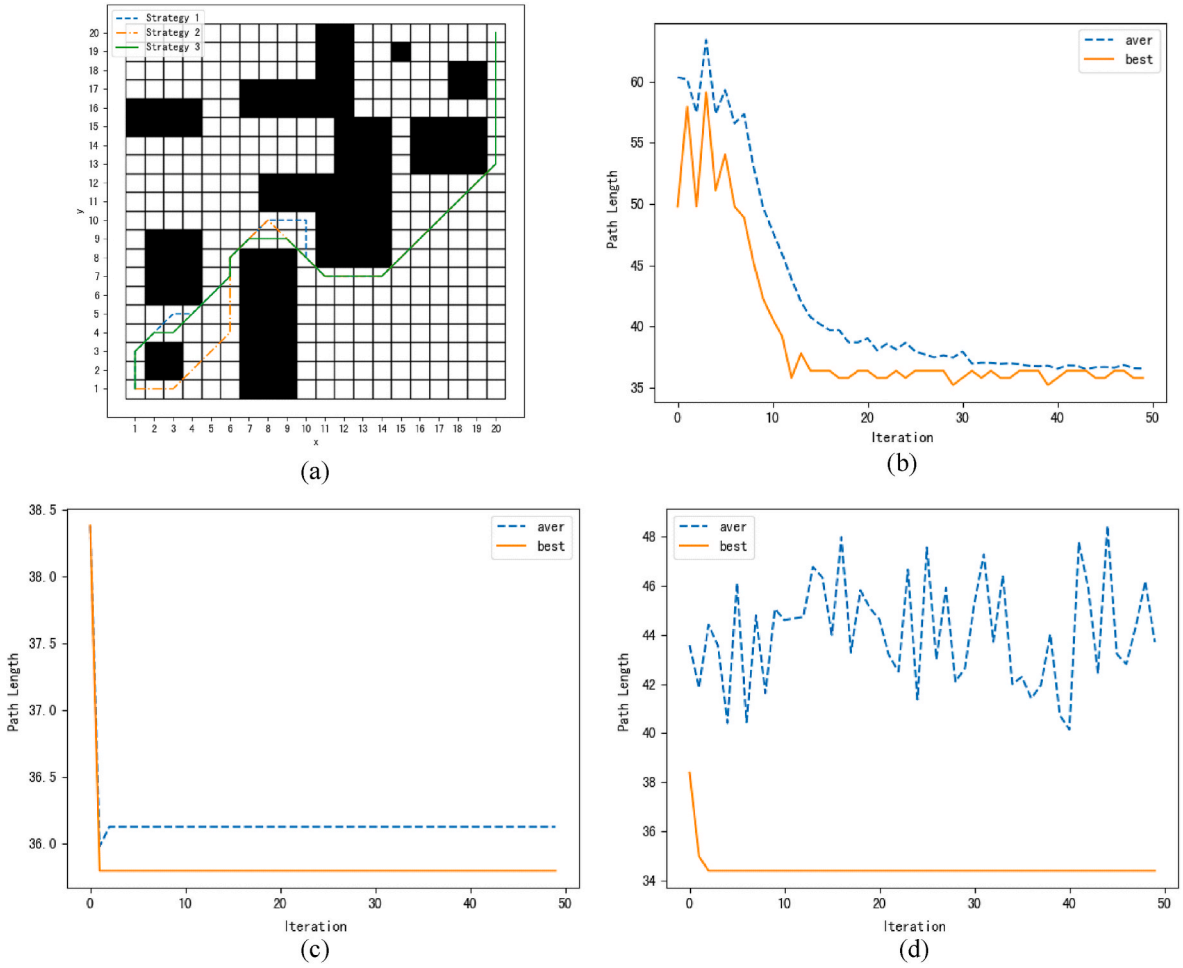


Fig. 6. Simulation results of three strategies. (a) Show the obtained paths of three strategies; (b), (c) and (d) are the iteration curves for strategy 1, strategy 2, and strategy 3, respectively.

Algorithm 2. IQACO

```
Procedure IQACO(map_data, start, end, max_iter, ant_num)
Init-Params: test map data map_data, start point start, end point end, maximum number
1:  of iterations max_iter, population size ant_num, Q-value matrix Q_Matrix that is equal in
    size to the map_data.
2:  For iteration from 1 to max_iter:
3:      Initialize success_way_list to an empty list;
4:      # Step 1: Each ant takes action sequentially
5:      For ant_no from 1 to ant_num:
6:          λ = ant_no/ant_num;
7:          Create an ant instance with Ant(start, end, λ , map_data, Q_Matrix);
8:          Run the ant with ant.run();
9:          If the ant successfully finds a path:
10:             Add the ant's path to success_way_list;
11:         End If
12:     End For
13:     # Step 2: Calculate the length of each path and update Q-value
14:     Initialize way_length_list to an empty list;
15:     For path in success_way_list:
16:         Initialize length to 0;
17:         For index from Len(path)-1 to 1 step -1:
18:             If Distance(path[index],path[index+1]) == 1:
19:                 γ = 0.9;
20:                 length += 1;
21:             Else:
22:                 γ = 0.85;
23:                 length += 2^0.5;
24:             Q_Matrix[path[index]] = max(γ *Q_Matrix[path[index+1]],
                Q_Matrix[path[index]]);
25:             End If
26:         End For
27:         Add the length to way_length_list;
28:     End For
29:     # Step 3: Summary for the current generation
30:     Calculate and store the average path length for the generation;
31:     Calculate and store the shortest path length for the generation;
32:     If a shorter path is found than the best known:
33:         Update the best known path length;
34:     End If
35: End For
End Procedure
Procedure Ant(start, end, λ , map_data, Q_Matrix)
```

5. Simulation experiments and analysis

In this section, to comprehensively evaluate the performance of the

```
Class Ant( ):
    def __init__(start, end, λ , map_data, Q_Matrix):
        Initialize input parameters as properties;
        Set the maximum step size of ant max_step is three times the size of map_data;
        Set the flag successful of ant pathfinding status to True;
        Set the path path of ant to an empty list;
    def run( ):
        Set current position position = start;
        For i from 1 to max_step:
            r = select_next_node( );
            If r == False:
                successful = False;
                break;
            Else If position == end:
                break;
            End If
        End For
        successful = False;
    def select_next_node( ):
        Calculate the list selectable_nodes of adjacent passable nodes;
        If selectable_nodes == None:
            Return False;
        End If
        If end in selectable_nodes:
            position = end;
            Add the positon to path;
            Return True;
        End If
        Initialize probability_list to an empty list;
        For node in selectable_nodes:
            Calculate the selection probability of nodes according to formula 7 and store
it in probability_list;
        End For
        If rand( )<1- λ :
            Use roulette wheel to select nodes based on the selection probability in
probability_list;
        Else:
            Select the node with the highest probability value based on the selection
probability in probability_list;
        End If
        Update position to selected node;
        Add the positon to path;
        Return True.
End Procedure
```

(*continued*).

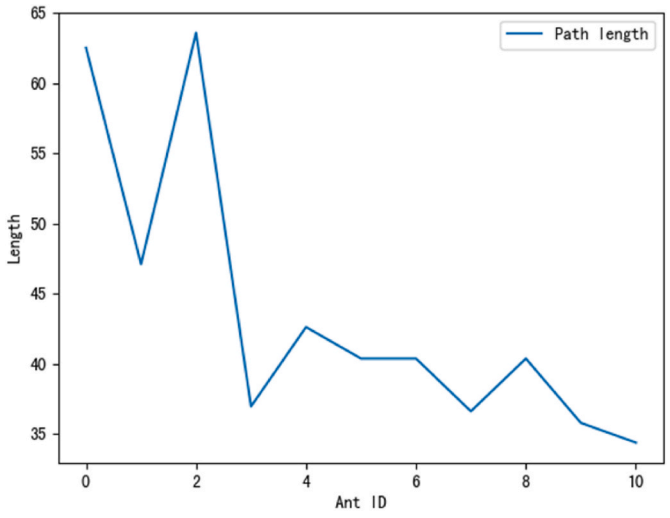


Fig. 7. The relationship graph between the path length obtained by a certain generation and the ant ID.

proposed IQACO algorithm, we first provide a theoretical analysis of its time complexity, supported by corresponding experimental results. Next, to assess the algorithm's robustness, we perform comparative tests between IQACO and traditional ACO across four different environments, followed by statistical analysis to evaluate the stability of IQACO. Finally, we compare IQACO with other state-of-the-art approaches to demonstrate its overall superiority.

Table 2
Parameters of IQACO.

Parameter	Value
Population size N	30
Max Iterations T	50
Q-value at the endpoint $Q(E)$	1

To ensure the reliability of the results, each experiment is independently repeated multiple times under the same conditions. Depending on the nature and objective of each experiment, we report a combination of descriptive statistics such as average, standard deviation, maximum, and minimum to illustrate the performance and consistency of the algorithm. These statistical measures are chosen to highlight both central tendency and variability, where applicable. While no formal hypothesis testing is conducted, the trends observed across repeated runs offer solid empirical evidence for the effectiveness of the proposed method.

5.1. Parameter selection

To validate the feasibility and effectiveness of the IQACO algorithm, a program was developed in Python and simulated on a PC. The hardware environment of the PC is as follows: the processor is a 13th Gen Intel(R) Core (TM) i7-13700K at 3.40 GHz, the RAM is 16 GB, and the system version is Windows 11. The default values of the relevant parameters of the IQACO algorithm in this section are shown in Table 2.

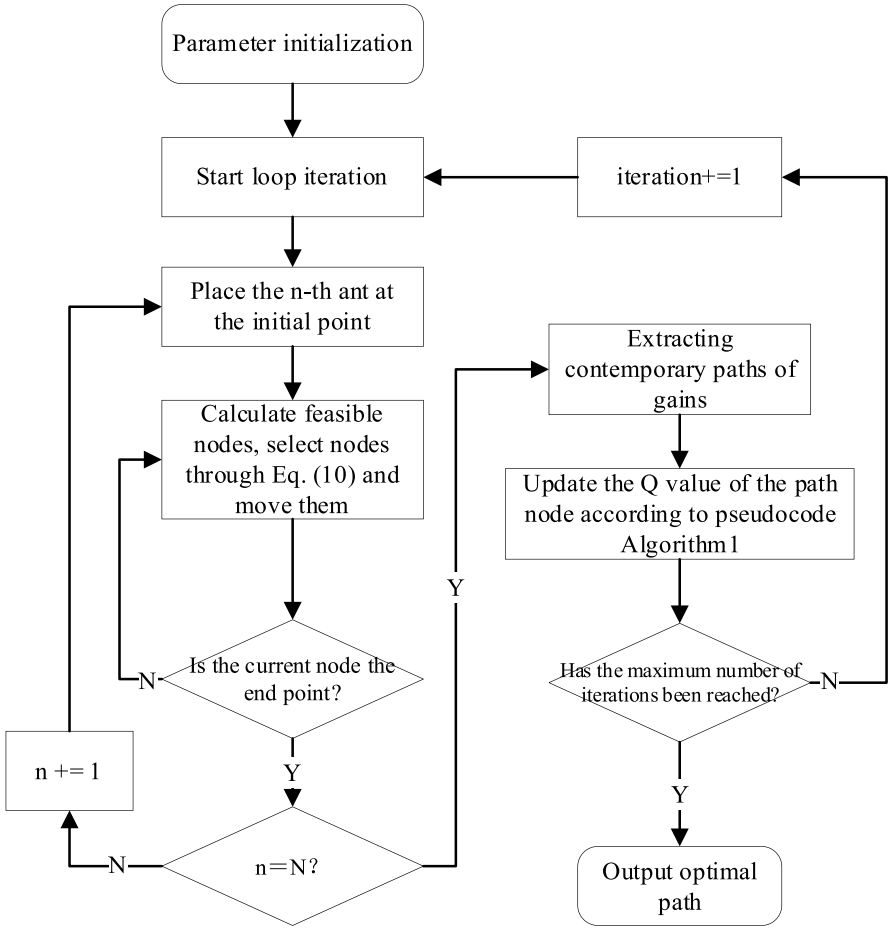


Fig. 8. The flow chart of IQACO.

Table 3
Running time data of two algorithms at different map size.

Algorithm	Map size	Population	Total running time/s			Running time of the first 10 generations/s		
			Max	Min	Average	Max	Min	Average
ACO	20 × 20	30	0.688118	0.666415	0.672933	0.106472	0.092693	0.098348
	30 × 30		1.099966	1.04871	1.072779	0.102083	0.08992	0.095978
	40 × 40		1.816942	1.713947	1.774088	0.190324	0.152153	0.171486
	50 × 50		2.321675	1.950325	2.05172	0.204931	0.183913	0.199234
IQACO	20 × 20	30	0.932444	0.920506	0.926561	0.098641	0.086184	0.092138
	30 × 30		1.074195	1.031374	1.058466	0.115296	0.09919	0.103496
	40 × 40		1.594771	1.507549	1.547297	0.159332	0.146010	0.150507
	50 × 50		1.732699	1.647349	1.682633	0.169346	0.154837	0.163502

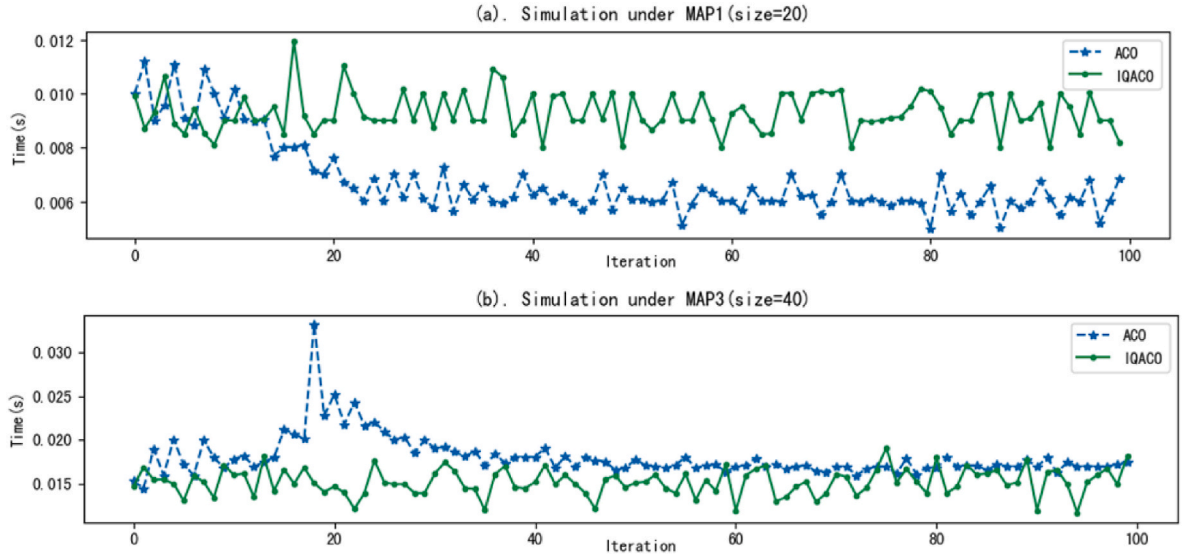


Fig. 9. The running time of each generation of the algorithm.

5.2. Time complexity analysis

5.2.1. Theoretical analysis

To analyze the time complexity of the IQACO algorithm, we need to consider the impact of its two innovations: the Q-evaluation strategy and the new node selection strategy, on the original algorithm. Firstly, the main impact of the Q-evaluation strategy on the original algorithm process occurs in the initialization step and the Q-value update step. On one hand, the time complexity of generating a Q-value for each node during initialization in IQACO is equivalent to the time complexity of generating a pheromone concentration value for each node in ACO. On the other hand, in the ACO algorithm, at the end of each generation, it is necessary to first solve the length of the obtained path, then calculate the additional pheromone concentration for each node and update it, and finally perform a volatilization operation for each node. In contrast, in the IQACO algorithm, as can be seen from Algorithm 2, the Q-value update and path length calculation can share the same loop, and no additional operations are needed for nodes that are not part of the current generation's path. Therefore, theoretically, the introduction of the Q-evaluation strategy can reduce the time complexity of the algorithm. Secondly, as can be seen from Algorithm 2, the new node selection strategy does not add extra selection operation steps to the IQACO algorithm.

In summary, the time complexity of the IQACO algorithm is essentially consistent with that of the original algorithm.

5.2.2. Experimental verification

To further demonstrate the advantage of IQACO in terms of running time, simulations were conducted using the traditional ACO algorithm and the proposed IQACO algorithm on four different map environments of varying sizes. The specific map settings are illustrated in Fig. 10. Each algorithm was independently simulated 20 times under identical conditions. For each algorithm and each map setting, we recorded two key indicators: the total running time over 100 iterations and the running time of the first 10 iterations. As shown in Table 3, we report the maximum, minimum, and average values of these indicators across the 20 runs. This statistical treatment aims to reduce the impact of randomness and ensure the reliability of performance evaluation.

Based on the data in Tables 3 and it is evident that when the map size is smaller (size of 20 × 20), the average total running time of ACO over 100 iterations is 0.672933 s, which is less than the 0.926561 s taken by the IQACO algorithm. This initially suggests a mistaken conclusion that the algorithm in this paper has longer running time. However, upon analyzing the running time of the first 10 iterations, it becomes apparent that IQACO completes in 0.092138 s, which is not significantly different from ACO's 0.098348 s.

To explain this phenomenon, the running time of each iteration of both algorithms over 100 iterations is statistically analyzed and compared, as presented in Fig. 9(a). Analysis of the data in the figure reveals that initially, during the early iterations, there is no significant difference in the running time per iteration between the two algorithms. This is because both algorithms are in the exploration phase, generating suboptimal paths, resulting in similar running time. As the traditional ACO algorithm gradually converges over iterations, the ant population begins to concentrate on the optimal path, leading to a significant decrease in the running time per iteration. In contrast, IQACO maintains effective search performance throughout the iterations. Therefore, in the later stages of iteration, while the path quality per iteration in IQACO is

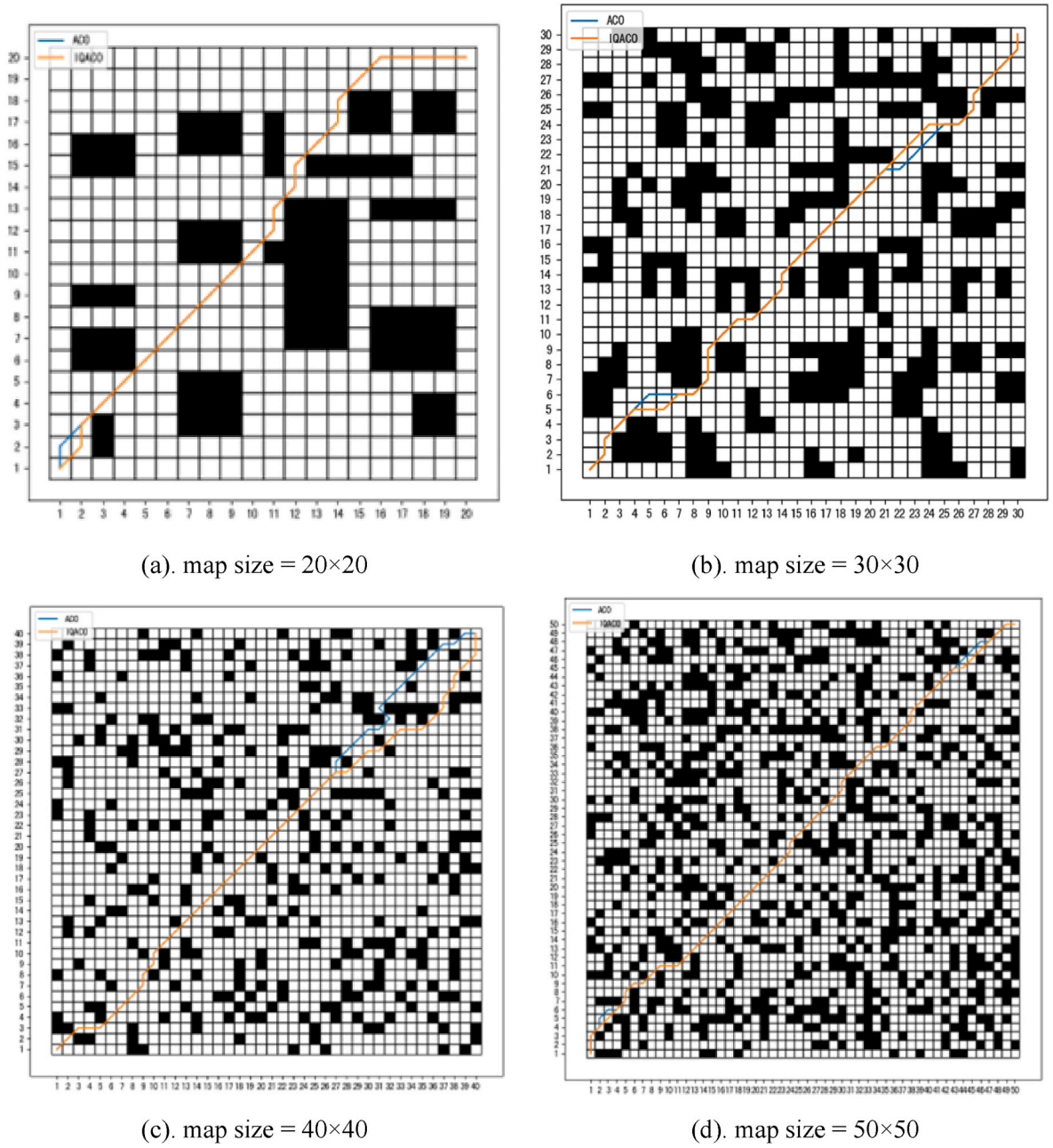


Fig. 10. The optimal path obtained by two algorithms at different map sizes.

generally inferior to that of the traditional ACO, the running time per iteration remains relatively stable. This trend in running time variation throughout the iterations is also reflected in Fig. 9(a).

As the map size increases, the traditional ACO algorithm gradually loses its search effectiveness. The first 10 generations' running times of the two algorithms were evaluated on map sizes of 30 × 30, 40 × 40, and 50 × 50, as summarized in Table 3. The results show that for the 30 × 30 map, the ACO algorithm and the IQACO algorithm have running times of 0.095978s and 0.103496s respectively, with no significant difference. However, in the 40 × 40 map environment, the IQACO algorithm runs in 0.150507s, slightly outperforming the ACO algorithm's time of 0.171486s. When the map size increases further to 50 × 50, the IQACO algorithm takes 0.163502s, which is noticeably better than the ACO algorithm's time of 0.199234s.

Fig. 9(b) further illustrates the iteration times of both algorithms per round on the 40 × 40 map. From the data in the figure, it is observed that the time for the ACO algorithm initially increases and then decreases. This phenomenon at the beginning of iterations primarily occurs because of insufficient search capability of the algorithm, resulting in some ants failing to reach the destination, thus fewer viable paths are found, and the running time per iteration decreases. As the concentration of pheromones on better paths increases, the success rate of the population's pathfinding rapidly improves, leading to a noticeable increase in running time per iteration. Furthermore, as the ant population gradually converges on suboptimal paths, the quality of paths obtained per iteration improves, shortening the pathfinding process for ants, thereby reducing the running time per iteration. However, due to the inadequate search performance at this stage, it is challenging for the ACO algorithm to converge to the global optimum, which fundamentally explains why the running time per iteration of the ACO algorithm remains higher than that of the IQACO algorithm in the later iterations. The data from Fig. 9 (b) confirms that the IQACO algorithm maintains a

Table 4
Simulation results of two algorithms at different map sizes.

Algorithm	Map size	Population	First generation optimal path		Convergence result		Minimum number of iterations	
			Average	Best	Average	Best	Average	Best
ACO	20 × 20	10	43.1131	40.0415	30.0100	**29.2131**	29	18
		30	40.1403	32.3847	29.5645	**29.2131**	20	13
		50	38.5646	35.7989	29.2131	**29.2131**	12	8
	30 × 30	10	/	/	/	/	/	/
		30	77.3038	68.7694	44.6569	**44.5268**	36	28
		50	74.5676	66.1836	44.5853	**44.5268**	28	18
	40 × 40	10	/	/	/	/	/	/
		30	120.7570	108.9531	61.2295	60.0831	49	40
		50	113.5904	95.4973	59.3155	58.9115	47	31
	50 × 50	10	/	/	/	/	/	/
		30	177.6312	153.5221	83.1352	78.4213	86	74
		50	157.0867	126.7104	76.8453	73.3968	84	76
IQACO	20 × 20	10	32.9444	29.7989	29.2131	**29.2131**	4	2
		30	30.2331	29.7989	29.2131	**29.2131**	3	2
		50	29.7989	29.7989	29.2131	**29.2131**	2	2
	30 × 30	10	50.9451	46.7694	44.5268	**44.5268**	11	3
		30	45.5552	45.3552	44.5268	**44.5268**	4	2
		50	45.3552	45.3552	44.5268	**44.5268**	3	2
	40 × 40	10	67.2369	61.7399	58.6689	**58.6689**	16	4
		30	63.8131	61.7399	58.6689	**58.6689**	5	2
		50	62.5399	61.1541	58.6689	**58.6689**	3	2
	50 × 50	10	96.5625	82.4678	72.8110	**72.8110**	24	10
		30	85.4820	78.2252	72.8110	**72.8110**	7	3
		50	83.4820	75.3968	72.8110	**72.8110**	5	3

consistently stable running time per iteration, confirming that the algorithm maintains stable search capability throughout the iteration cycle.

This section presents the characteristics of the proposed IQACO algorithm in terms of runtime through both theoretical analysis and simulation experiments. Specifically, firstly, under the same exploration performance, the proposed IQACO algorithm shows a clear advantage in runtime compared to the traditional ACO algorithm. This advantage arises because, in each iteration, the traditional ACO algorithm involves two loops: one for calculating the path length and another for updating the pheromone concentration on nodes. In contrast, the IQACO algorithm updates the Q-values while calculating the path length, thereby eliminating one loop. Secondly, in environments with maps of varying sizes, the runtime of the IQACO algorithm remains relatively constant throughout the entire iteration period, thanks to its consistently good exploration performance. This characteristic is attributed to the node selection strategy introduced in Section 4.2, which ensures population diversity by assigning each individual in the population a varying degree of sensitivity to the Q-value, thereby maintaining the stability and efficiency of the algorithm.

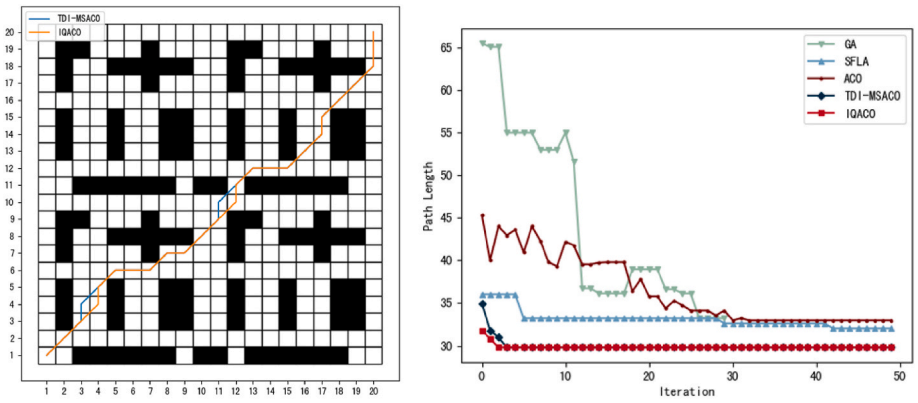
5.3. Algorithm robustness experiment

Based on the comparison between Eq. (7) and Eq. (8), it is evident that IQACO requires fewer parameters than the traditional ACO. This theoretically implies that the proposed algorithm should demonstrate greater robustness when applied to environments with different map sizes. To verify this assumption, simulations were performed using IQACO and the traditional ACO on map environments with sizes of 20 × 20, 30 × 30, 40 × 40, and 50 × 50.

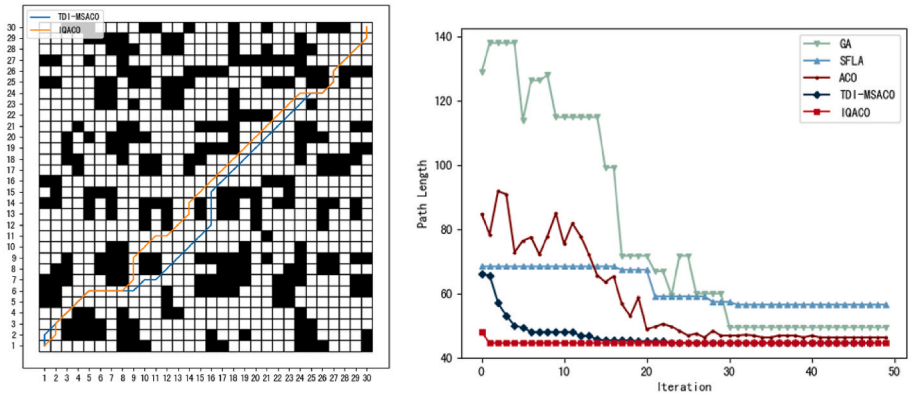
To ensure statistical reliability and minimize the influence of randomness, each experiment was repeated 20 times under consistent conditions. The results are presented in Table 4, where three key performance indicators are reported. These include the optimal path length obtained in the first generation, the convergence result, and the minimum number of iterations required for convergence. For each of these indicators, both the average value and the best value across the 20 repetitions were calculated and reported. This statistical treatment allows for a more comprehensive assessment of the algorithm's stability across different environments. In addition, to explore the impact of varying population sizes, further experiments were conducted with the number of ants set to 10, 30, and 50. The corresponding simulation results for the optimal paths obtained by the two algorithms at different map sizes are illustrated in Fig. 10.

Based on the analysis of the data in Table 4, several conclusions can be drawn. Firstly, due to its numerous adjustable parameters, traditional ACO exhibits significant sensitivity to the size of the environment. In the experiments mentioned, traditional ACO performs well on map sizes of 20 × 20 and 30 × 30, but its effectiveness notably declines as the map size increases further. This is evidenced by the significant increase in initial best solution values, indicating increased pressure during the generation of initial solutions. Moreover, when the number of ants is low, traditional ACO may struggle to generate feasible initial solutions, thereby failing to complete the optimization task. In contrast, the IQACO algorithm primarily adjusts the population size parameter. Its fewer adjustable parameters enable it to consistently perform well across different environments. The data in Table 4 also confirms that IQACO achieves optimal solutions in all four tested environments. Therefore, the analysis suggests that IQACO's performance stability across varying environmental conditions is a notable advantage over traditional ACO, which is highly sensitive to its parameter settings, particularly in larger-scale environments.

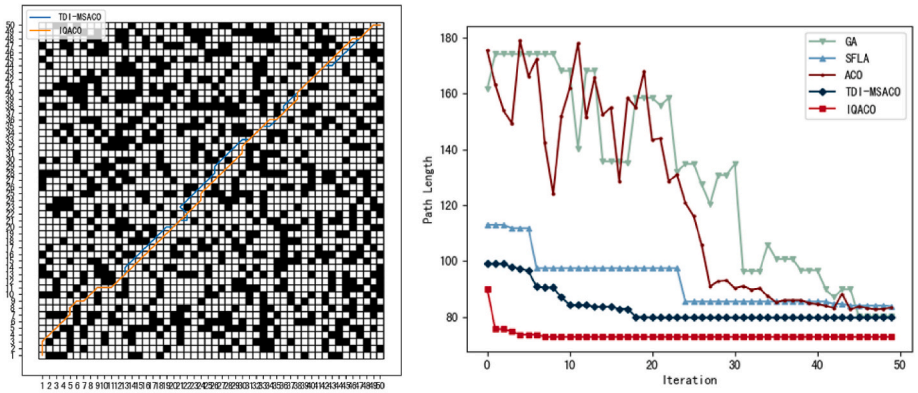
Secondly, from the data in the table above, it can be observed that increasing the population size in both traditional ACO and IQACO algorithm can enhance algorithm stability. However, it is noteworthy that in traditional ACO, increasing the population size does not significantly improve optimization efficiency. This is because although a larger population size can provide more feasible solutions and improve algorithm stability, it also introduces more non-optimal path solutions, leading to increased interference from ineffective pheromone concentrations. Therefore, adjusting the population size to enhance the performance of traditional ACO algorithm is not an effective approach. In contrast, for the IQACO algorithm, the data from the table shows that when the population size is 10, the algorithm can obtain optimal solutions, but the solution finding efficiency is influenced by the map size. For example, in a 20 × 20 map, the average minimum iteration counts to achieve the optimal solution is only 4. However, in environments of 30 × 30, 40 × 40, and 50 × 50, the minimum iteration count gradually



(a). Simulation results in the 20×20 environment



(b). Simulation results in the 30×30 environment



(c). Simulation results in the 50×50 environment

Fig. 11. Simulation results in three different environments.

increases to 11, 16, and 24, respectively. When the population size is increased to 30, the algorithm's performance improves significantly. Further increasing the population size to 50 does not yield additional improvements in algorithm performance, demonstrating that the optimal comprehensive performance of the algorithm is achieved with a population size of 30.

In summary, the experiments in this section demonstrate that the proposed IQACO algorithm exhibits strong robustness and adaptability across different environmental scales. Unlike traditional ACO, which is highly sensitive to parameter settings and shows degraded performance as the map size increases, IQACO significantly reduces the number of tunable parameters and, more importantly, decouples these parameters from the complexity of the environment. This allows IQACO to maintain stable optimization performance without frequent parameter adjustments. Furthermore, while increasing the population size enhances the stability of both traditional ACO and IQACO, only IQACO effectively translates this improvement into better optimization results. These findings validate the robustness and scalability of the proposed algorithm under varying environmental conditions.

5.4. Algorithm superiority experiment

5.4.1. Experiment comparisons with related work

In literature (D. Li et al., 2023), Li et al. introduced a more efficient heuristic measure, the terminal distance index (TDI), to evaluate node quality in the ACO algorithm, aiming to replace the role of pheromone concentration. Eq. (11) presents the new node selection probability formula proposed by them after introducing the terminal distance index.

However, it is observed that the introduction of the termination distance index did not reduce the number of hyperparameters in the algorithm. As a result, the algorithm still exhibits instability in searching across different map sizes.

Moreover, during initialization, each node requires a sufficiently large termination distance index. As iterations progress, significant disparities in termination distance indices among neighboring nodes emerge after updates, as indicated by Eq. (11). These large differences propagate into node selection probabilities, causing confusion in node quality assessment and thereby affecting the optimization performance of the algorithm.

In contrast, observing the node selection probability formula Eq. (8) in IQACO, we note that part $\lambda\cdot(1+Q(j))$ influences the selection probability $p_{ij}^{n}(t)$ based on node Q-value. Here, λ is an adaptive coefficient related to population size, unaffected by the node's Q-value. Any node j has a Q-value $Q(j)$, ranging from 0 to 1. Consequently, $1+Q(j)$ ranges from 1 to 2. This constraint ensures that throughout the entire iteration cycle, the influence of Q-value on node selection probability remains within a small range, avoiding drastic fluctuations. Moreover, this advantage remains effective across experiments conducted in environments of different sizes.

$$p_{ij}^{k}(t)=\begin{cases}\dfrac{\left[\tau_{ij}(t)\right]^{\alpha}\cdot\left[1/\mathrm{k}_{ij}\right]^{\beta}}{\sum\limits_{s\in\ allowed_k}\left[\tau_{ij}(t)\right]^{\alpha}\cdot\left[1/\mathrm{k}_{ij}\right]^{\beta}}, & s\ \varepsilon\ allowed_k\\ 0, & s\notin allowed_k\end{cases} \tag{11}$$

To demonstrate the superiority of our approach, we chose the simulation environments depicted in Figures 16, 17, and 18 from reference (D. Li et al., 2023), which represent environments of size 20 × 20, 30 × 30, and 50 × 50, respectively. In these environments, we conducted experimental simulations comparing the Genetic Algorithm (GA), Shuffled Frog-Leaping Algorithm (SFLA), Ant Colony Optimization (ACO), our proposed IQACO algorithm, and the TDI-MSACO algorithm proposed in the literature.

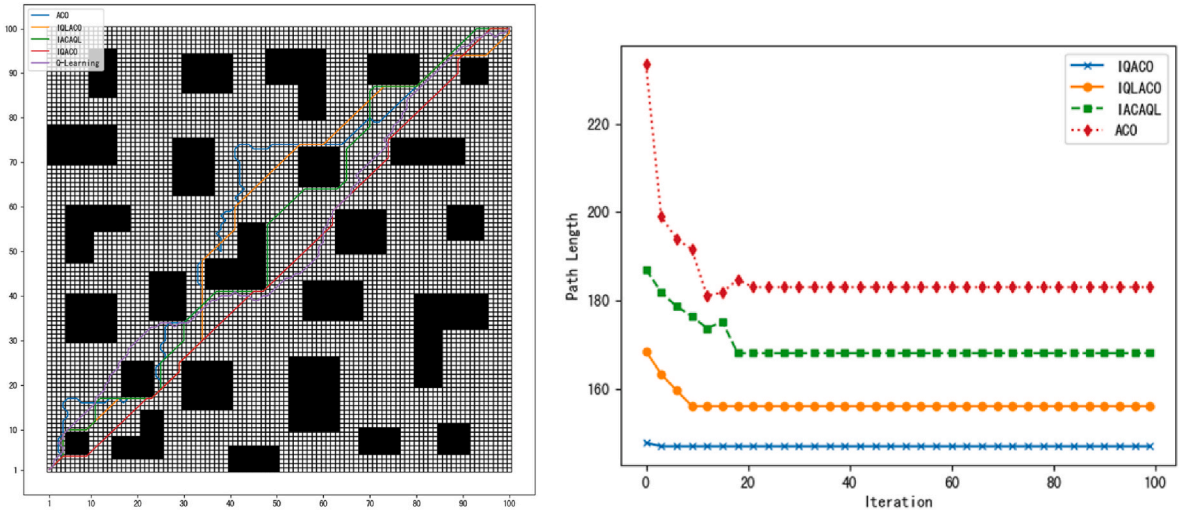
In reference (D. Li et al., 2023), Li et al. utilized a multi-step strategy to enhance the search capability of the TDI-MSACO algorithm. To ensure fairness in our experiments, we set the ant step size to 1, following the parameters reported in the literature, while keeping other parameters consistent. The statistical results of our experiments in three environments of varying sizes are shown in Fig. 11, including the path planning performance and convergence behavior of different algorithms.

In the 20 × 20 environment, according to the data from Fig. 11 (a), the initial optimal solutions of the IQACO and TDI-MSACO are 31.7989 and 34.9705, respectively. These values compare favorably to the initial solutions of GA, SFLA, and ACO algorithms, which are 65.5269, 36.04, and 42.1421, respectively, demonstrating significant advantages. Moreover, the IQACO and TDI-MSACO algorithms both found the optimal solution of 29.7989 in the 3rd and 4th generations, whereas the other algorithms not only iterate slower but also fail to converge to the optimal solution.

In the 30 × 30 environment, according to the data from Fig. 11 (b), the initial solution of IQACO is 47.9410, significantly better than the initial solution of TDI-MSACO from the literature, which is 65.9411. Both values are superior to several other comparative algorithms, with the best solution among them being 68.4321 from the SFLA algorithm. Regarding the convergence solutions, only the IQACO and TDI-MSACO algorithms converge to the optimal solution of 44.5268. The convergence generation for TDI-MSACO is 14, which is worse than the 2 generations required by the IQACO algorithm.

In the 50 × 50 environment, based on the data from Fig. 11 (c), the initial solutions of the GA, SFLA, ACO, and TDI-MSACO algorithms are 161.5696, 136.0243, 175.4799, and 99.0831 respectively. The initial solution of the IQACO algorithm is 89.8820, which is superior to the initial solutions of the other algorithms. Moreover, the IQACO algorithm converges in 8 iterations, which is better than TDI-MSACO's convergence in 18 iterations and also superior to the iteration counts of the other three algorithms, which are 24, 45, and 48 respectively. Furthermore, among all simulation algorithms, only the IQACO algorithm achieves the optimal solution of 72.8110.

Based on the above analysis, it can be observed that when the environment size is small, such as on the 20 × 20 map, there is no significant difference in the performance of TDI-MSACO and IQACO algorithms in terms of initial solution quality or convergence speed. This is because at this scale, the pathfinding method of ACO can still achieve good results, allowing TDI-MSACO to also exhibit good pathfinding effectiveness. However, as the map size increases, such as on the 30 × 30 map, the efficiency of the ACO algorithm's pathfinding method begins to show slight inadequacy. Consequently, the quality of initial solutions notably decreases, which is reflected in the initial solutions of MDI-MSACO. It can be observed that the initial solutions of TDI-MSACO are significantly inferior to IQACO at this stage. Nevertheless, TDI-MSACO still converges to the optimal solution, albeit requiring more iterations. When the environment size further increases to 50 × 50, TDI-



(a). Optimal paths (b). Convergence curves

Fig. 12. Optimal paths and convergence curves of algorithms in the 100 × 100 grid environment.

Table 5
Comparison of simulation results of four algorithms.

Algorithms	Path length			Iteration times			Time consumption/s	p-value
	Best	Mean	SD	Best	Mean	SD		
Q-Learning	161.0362	163.8764	2.96	–	–	–	8.61	2.8e-8
ACO	182.8234	185.8474	3.60	18	19.5	1.84	3.23	1.2e-9
IQLACO	156.1665	157.1911	1.14	8	9.1	0.99	8.61 + 1.59	4.5e-6
IACAQL	168.1249	170.4620	2.66	19	20.6	1.84	3.38	4.3e-8
IQACO	**147.0363**	147.0363	0	**2**	2.65	0.48	0.3	–

MSACO and ACO not only exhibit inferior initial solution quality compared to IQACO, but they also fail to converge to the optimal solution even after more iterations. In contrast, IQACO consistently produces the best initial solutions across all environments and rapidly converges to the optimal solution.

In summary, while TDI-MSACO extends traditional ACO through the incorporation of a terminal distance index, its failure to reduce parameter complexity and the instability in node evaluation limit its effectiveness to small-scale environments. In contrast, IQACO achieves superior generalization by significantly reducing the number of parameters and employing a Q-value-based node selection mechanism to preserve population diversity. Its consistently strong performance across varying environment sizes effectively addresses the limitations of existing approaches and validates the robustness and applicability of the proposed method.

5.4.2. Experiment comparisons with Q-learning-enhanced ACO algorithms

To further demonstrate the superiority of the proposed IQACO algorithm over existing Q-learning-enhanced ACO algorithms, we compare it with two recent representative algorithms published in the past two years: an improved ant colony Q-learning algorithm (IQLACO) (Cui et al., 2025) and an improved ant colony algorithm based on Q-Learning (IACAQL) (Zhao et al., 2024). In addition, the standard Q-Learning algorithm is incorporated as a baseline to highlight the advantages of integrating Q-values into the ACO framework.

In the IQLACO algorithm, Q-learning is used to pre-train the initial pheromone distribution in ACO. Although early-stage search performance is improved through this strategy, additional computational overhead is introduced during the pre-training process, which compromises the algorithm's real-time adaptability. Furthermore, the Q-learning-based pheromone distribution, once fixed, may dominate the search process and reduce exploration capability, increasing the risk of premature convergence. In contrast, the IACAQL algorithm replaces the pheromone update mechanism with a Q-value learning process. In this method, pheromone increments are directly embedded into the reward term of the Q-value update, and the use of action expectations is omitted. As a result, the improvement in guidance quality is limited compared to that achieved by conventional pheromone mechanisms.

In comparison, the proposed IQACO algorithm introduces a node-level Q-value evaluation strategy to facilitate efficient solution assembly. By transforming the direct search for the optimal solution into an indirect construction based on sub-optimal segments, the algorithm significantly reduces the overall search complexity.

Table 6
The sources of simulation maps.

Map Name	Map size	Literatures for the map sources and figure numbers
MAP1	20 × 20	literature (Shi et al., 2022), Fig. 8
MAP2	20 × 20	literature (Wu et al., 2023), Figure 28
MAP3	20 × 20	literature (Miao et al., 2021), Fig. 8
MAP4	30 × 30	literature (Cui et al., 2022), Fig. 6
MAP5	30 × 30	literature (Chen et al., 2022), Fig. 7
MAP6	50 × 50	literature (Chen et al., 2022), Fig. 8

The following experiments are conducted to further validate the superiority of the proposed algorithm. In the experiments, the common parameters of all ACO-based algorithms are kept consistent with those in the literature (Cui et al., 2025), while the remaining parameters are set according to their respective original papers. The test case is based on the 100 × 100 map environment shown in Fig. 10 of the literature (Cui et al., 2025). All algorithms are executed continuously for 20 runs. For the Q-Learning algorithm, the parameter settings follow the configuration used in the literature (Pan et al., 2022), where the number of attempts is set to 3000, the learning rate is 0.8, the discount factor is 0.5, and the exploration factor is 0.3. Fig. 12 illustrates the optimal paths and convergence curves of different algorithms with 100 iterations in the 100 × 100 grid environment, while Table 5 provides a detailed comparison of their simulation results, including the path length and iteration times reported as best, mean, and standard deviation, as well as time consumption reported as the average.

It is worth noting that the Q-Learning algorithm is not included in the convergence curve comparison (Fig. 12(b)), as its iteration mechanism fundamentally differs from that of ACO-based algorithms. Specifically, Q-Learning performs step-by-step updates based on a single agent's actions, whereas ACO-based algorithms operate in generations where each iteration involves a population of agents. Due to this inconsistency in iteration granularity, the convergence curves of Q-Learning are not directly comparable and are therefore omitted from the figure.

As shown by the experimental data in Fig. 12 and Table 5, IQLACO

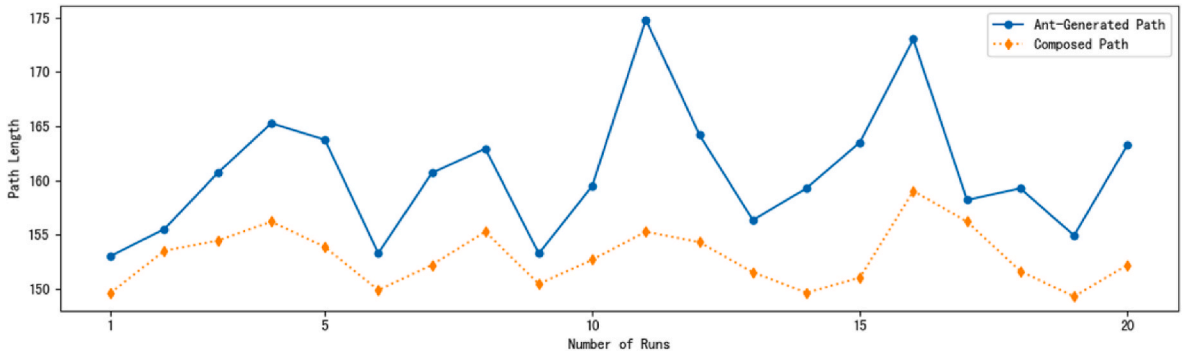


Fig. 13. Comparative analysis of ant-generated and composed optimal path lengths over 20 IQACO runs.

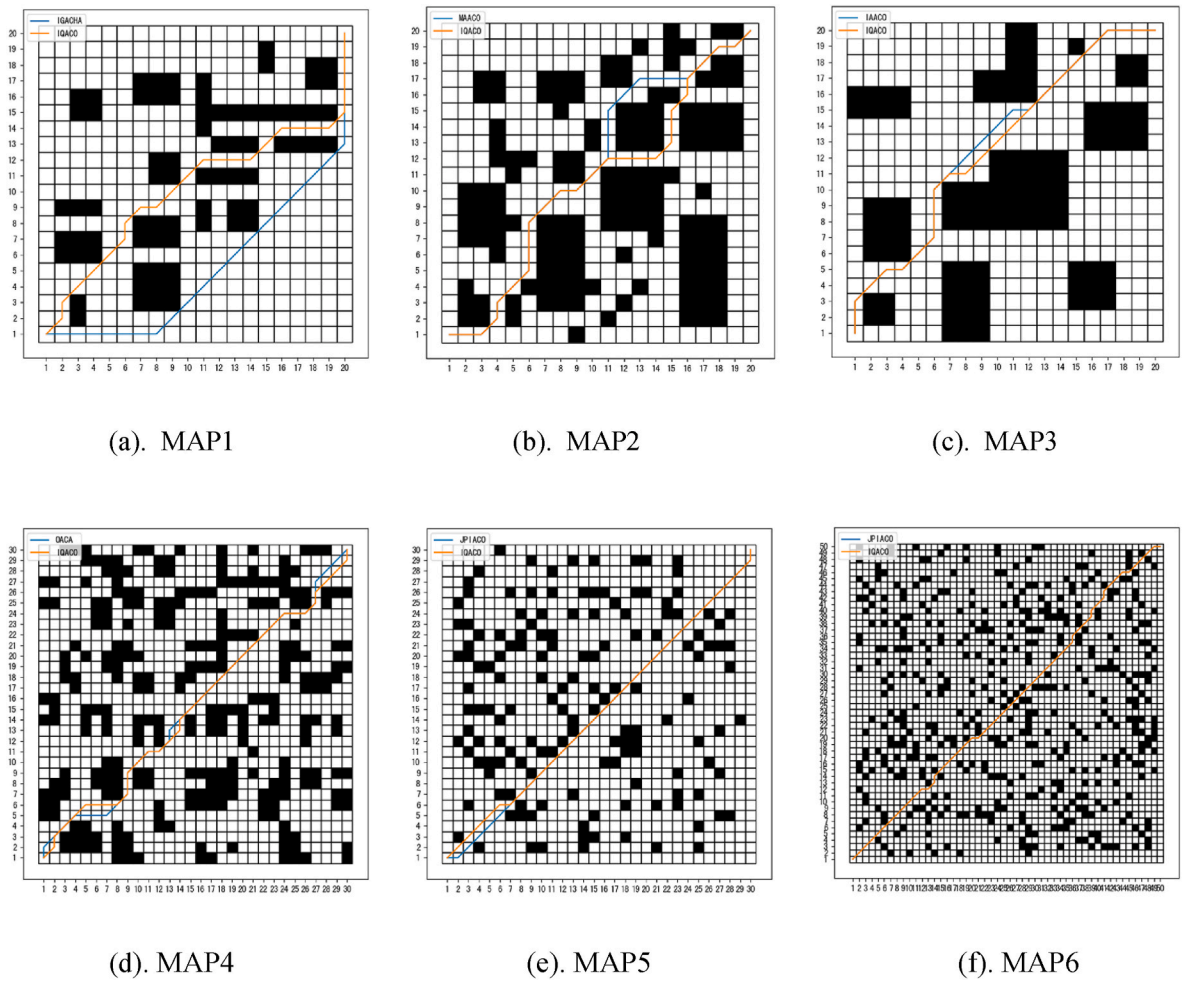


Fig. 14. Paths obtained by algorithms in 6 different environments.

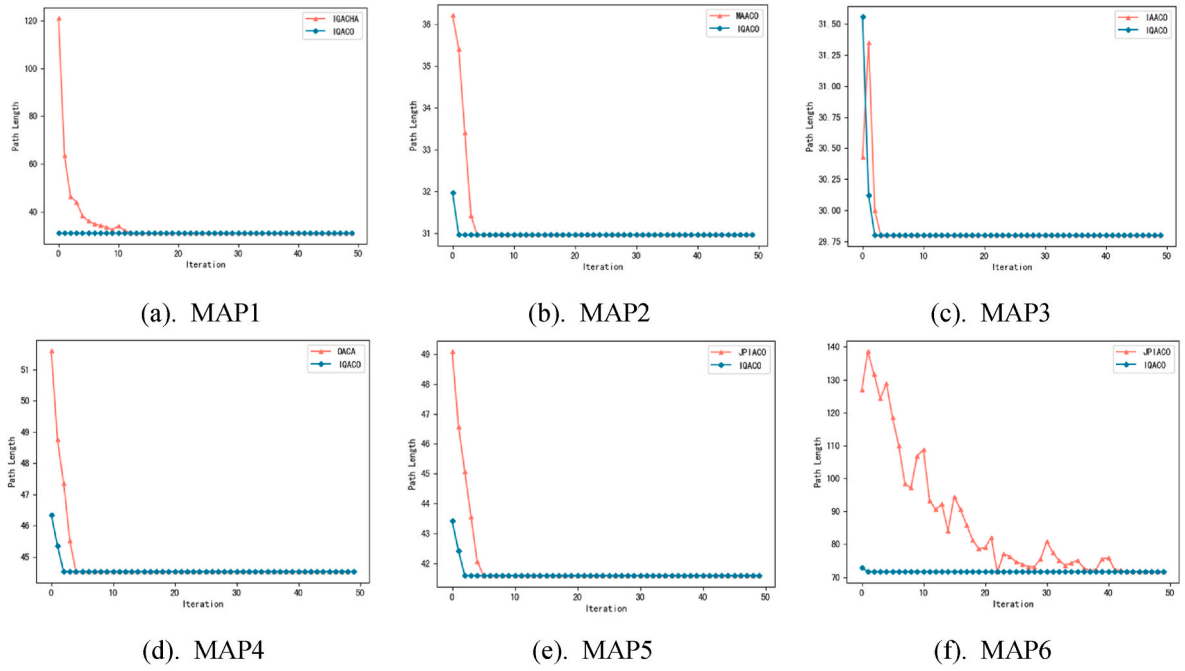


Fig. 15. Iteration curves of algorithms in 6 different environments.

Table 7
Experimental result.

Map Name	Algorithms	Path length			Iteration times			p-value
		Best	Mean	SD	Best	Mean	SD	
MAP1	ACO	**30.9705**	31.6712	0.9	18	21	1.4	2.1×10^{-5}
	IGACHA	**30.9705**	31.2105	0.9	13	16	0.9	3.6×10^{-4}
	IQACO	**30.9705**	30.9705	0	**1**	1.4	0.3	–
MAP2	ACO	32.7279	34.5214	1.2	30	36	1.6	2.4×10^{-7}
	IPACO	**30.9705**	30.9705	0	5	5.3	0.2	1.2×10^{-7}
	IQACO	**30.9705**	30.9705	0	**2**	2	0	–
MAP3	ACO	30.9705	36.6090	3.4	14	23	6.8	6.5×10^{-6}
	IAACO	**29.7989**	30.1220	0.4	5	6.4	1	4.1×10^{-5}
	IQACO	**29.7989**	29.7989	0	**3**	4.8	1.6	–
MAP4	ACO	48.2341	55.1520	5.4	41	46	5.5	3.9×10^{-8}
	QACA	**44.5268**	45.2912	0.7	6	16	5.3	6.7×10^{-6}
	IQACO	**44.5268**	44.5268	0	**2**	2.8	0.8	–
MAP5	ACO	44.4180	45.3241	0.9	36	41	4.2	8.3×10^{-6}
	JPIACO	**41.5978**	41.8341	0.3	6	9	0.6	4.5×10^{-5}
	IQACO	**41.5978**	41.5978	0	**2**	3.2	0.4	–
MAP6	ACO	112.4840	128.2301	9.8	42	48	5.3	2.1×10^{-7}
	JPIACO	**71.6394**	71.9241	0.6	22	28	2.7	1.1×10^{-5}
	IQACO	**71.6394**	71.6394	0	**2**	4.1	0.7	–

significantly improves the quality of the initial solution by introducing Q-Learning pre-trained pheromone concentrations and leveraging the experience from the basic Q-Learning algorithm. However, due to the uneven distribution of the initial pheromone, the algorithm's exploration ability is reduced, which accelerates the convergence and ultimately causes it to fall into a local optimum (156.1665). In terms of runtime, excluding the 8.61 s required for Q-Learning pre-training, IQLACO takes 1.24 s to obtain the optimal solution, which appears to indicate efficient problem-solving. Nevertheless, in dynamically changing environments, the high time cost of QL pre-training presents a significant challenge to IQLACO's real-time applicability.

In contrast, the IACAQL algorithm introduces a new pheromone update strategy, which moderately enhances the search capability and achieves a local optimal solution (168.1249). However, its convergence speed remains similar to that of the traditional ACO algorithm, primarily because the underlying pheromone guidance mechanism has not undergone fundamental changes.

The IQACO algorithm proposed in this paper strengthens optimization performance by combining multiple suboptimal solutions to form a better solution. It successfully finds the global optimum (147.0363) in all 20 independent runs, demonstrating excellent stability. With an average of only 2.65 iterations required, the algorithm also exhibits high efficiency. Moreover, IQACO achieves the optimal solution in just 0.3 s, highlighting its superior real-time performance.

To further verify the observed performance advantages, independent two-sample *t*-tests were conducted on the path length results over 20 runs. The results show that IQACO significantly outperforms all baseline algorithms, including IQLACO and IACAQL, with *p*-values well below 0.05. These outcomes confirm that the superiority of IQACO is not only consistent in empirical observations but also statistically significant.

Additionally, as shown in Fig. 13, which compares ant-generated and composed optimal path lengths over 20 IQACO runs, although the shortest path found by the ants in the first generation is not ideal, the algorithm consistently constructs a better path through path composition. This validates the effectiveness and feasibility of the proposed innovation that combines suboptimal paths to derive a more optimal solution.

In summary, although both IQLACO and IACAQL achieve some performance improvements by incorporating Q-learning mechanisms, the former enhances initial information guidance through pre-training, and the latter improves local search quality by replacing pheromone updates with Q-values. However, both still suffer from issues such as insufficient exploration ability, poor convergence stability, or inadequate real-time performance. In contrast, the proposed IQACO algorithm significantly reduces search complexity through a path combination mechanism. This mechanism is realized through the node Q-update strategy introduced in Section 4.1.2. At the same time, the algorithm maintains solution quality and balances the stability and real-time performance of the optimization process. In all experiments, IQACO consistently finds the global optimal solution, fully validating the effectiveness and superiority of the proposed method.

5.4.3. Experiment comparisons with recent work

To further validate the innovation and superiority of the proposed IQACO algorithm, six simulation environments were selected, as detailed in Table 6, which lists the source references for the test maps. We conducted comparative experiments between IQACO and several representative algorithms from the literature. The optimal paths generated by each algorithm in the six environments are illustrated in Fig. 14, and the corresponding iteration curves are shown in Fig. 15. The statistical results, including the best, mean, and standard deviation values of the path length and the number of iterations required to reach the optimal solution, are summarized in Table 7.

Based on the data in Tables 6 and it is evident that in three 20 × 20 environments, the IQACO slightly outperforms traditional ACO and algorithms found in literature in terms of solution quality. However, in terms of solution stability, the IQACO shows significant advantages. It is observed that the standard deviation (SD) of the optimal solutions across the three environments is 0. Moreover, in terms of search speed, convergence is generally achieved within 10 generations, which is markedly better than traditional ACO and literature algorithms.

In two 30 × 30 environments, the performance of the traditional ACO notably deteriorates, with the minimum convergence generation significantly increasing. Conversely, both the literature algorithm and the IQACO continue to find optimal paths. However, in terms of stability, the literature algorithm exhibits some fluctuation in solution quality, whereas the IQACO algorithm demonstrates consistently outstanding stability in finding solutions. Despite a slight impact on search speed compared to the literature algorithm, the IQACO algorithm's minimum iteration count remains relatively unaffected and superior to that of the literature algorithm.

In the 50 × 50 environment, the traditional ACO nearly loses its ability to find optimal solutions, while the literature algorithm, although capable of achieving optimal solutions, exhibits further decreased stability and significantly slower convergence. On the other hand, the IQACO maintains excellent stability in finding solutions while ensuring optimal results. Although there is some increase in search speed, it remains markedly better than the literature algorithm. The above

experimental results demonstrate the superiority of the IQACO algorithm proposed in this paper compared to recent research findings.

Independent two-sample *t*-tests were also conducted on the path length results between IQACO and the best-performing comparative algorithm in each environment. As summarized in Table 7, all resulting *p*-values are well below the standard significance threshold of 0.05. This confirms that the observed improvements in path quality by IQACO are statistically meaningful across different map sizes, rather than outcomes of random variation.

In summary, experimental results across various map sizes demonstrate that the proposed IQACO algorithm exhibits significant advantages over traditional ACO and recent literature-based approaches. In 20 × 20 environments, although IQACO only slightly outperforms alternative methods in terms of solution quality, it stands out with exceptional solution stability—achieving zero standard deviation and rapid convergence within few generations. In 30 × 30 environments, traditional ACO suffers a notable performance decline, and while the literature algorithm can still obtain optimal solutions, its stability is subject to fluctuation; in contrast, IQACO consistently delivers robust performance with superior convergence speed. In 50 × 50 environments, traditional ACO nearly fails to find optimal solutions, and the literature algorithm further loses stability and converges much slower. Conversely, IQACO maintains excellent stability and optimality, converging markedly faster. These results validate the superior robustness, efficiency, and adaptability of IQACO, clearly demonstrating its comprehensive performance improvements compared to recent approaches.

6. Management-level impact analysis

The IQACO algorithm proposed in this study significantly improves the performance and adaptability of path planning by introducing a Q-value-based node evaluation mechanism along with a dynamic Q-value update strategy. These algorithmic enhancements allow for better combination of suboptimal paths and maintain strong exploration capability throughout the optimization cycle. In practical manufacturing settings—particularly in shop-floor environments where AGVs rely on predefined magnetic tracks composed of straight and turning segments—the grid-based modeling approach adopted in this work aligns well with real deployment constraints. This ensures that the proposed algorithm is not only theoretically sound but also practically feasible. From a managerial perspective, the improved robustness, convergence speed, and solution quality offered by IQACO can directly contribute to more reliable AGV scheduling, reduced planning time, and lower energy consumption. These benefits, in turn, help manufacturing enterprises enhance overall resource utilization, reduce operational costs, and improve system resilience, especially in complex or dynamic shop-floor environments. The proposed method thus provides a promising foundation for supporting intelligent logistics and execution systems in the era of smart manufacturing.

7. Conclusion and future research

To address the limitations of traditional ant colony optimization (ACO) algorithms in path planning, particularly their reliance on pheromone concentration for node evaluation and the instability caused by manually adjusted heuristic coefficients, this paper proposes an improved Q-evaluation ant colony optimization (IQACO) algorithm. The approach is inspired by the principles of Q-learning and introduces a Q-value-based node evaluation mechanism to replace traditional pheromone-guided strategies. This helps reduce the sensitivity to hyperparameters and improves the stability of the decision-making process. A method for converting path transitions into Q-values is designed and verified, along with a Q-evaluation-based node selection strategy that enhances decision accuracy and exploration performance.

Extensive experiments conducted in grid-based static environments demonstrate that IQACO consistently outperforms not only traditional ACO but also several recent variants of ACO and ACO-QL algorithms in terms of optimization performance, exploration capability, and time efficiency. Theoretical analysis and complexity verification further confirm the efficiency of the proposed algorithm. Robustness is demonstrated through repeated tests in multiple map scenarios.

It is also important to note that the IQACO algorithm is fundamentally built upon the Q-learning framework, with all strategies relying on node-centric Q-value estimation. Therefore, its applicability depends on whether the target problem can be modeled as a Markov Decision Process (MDP). Typical application areas include path planning, the traveling salesman problem (TSP), and scheduling problems in discrete manufacturing systems, where state transitions follow the Markov property. In environments that deviate significantly from MDP assumptions, additional adaptation may be required to maintain performance.

Although the current study reports performance using maximum, minimum, average, and standard deviation metrics to illustrate the effectiveness and stability of the proposed algorithm, more rigorous statistical significance testing remains to be conducted. In future work, such methods will be employed to further enhance the reliability and generalizability of experimental comparisons. In addition, future research will focus on expanding the algorithm's applicability to more dynamic and uncertain environments. This includes integrating the algorithm with real-time data acquisition and decision-making frameworks, deploying it in real-world scenarios such as dynamic manufacturing shop floors, and verifying its effectiveness using actual industrial equipment such as automated guided vehicles (AGVs). Furthermore, we aim to explore multi-agent extensions by enabling the sharing of partial Q-value information among agents to enhance cooperative perception and learning. Combining reinforcement learning techniques with heuristic methods, for example through deep Q-networks to approximate Q-values in large-scale or continuous environments, also offers promising directions for improving performance in complex industrial applications.

To facilitate reproducibility and future research, the complete source code is publicly available at: https://github.com/LiDongDong1996/IQACO.

CRediT authorship contribution statement

Dongdong Li: Writing – original draft, Visualization, Validation, Supervision, Software, Formal analysis, Data curation, Conceptualization. **Lei Wang:** Writing – review & editing, Resources, Project administration, Methodology, Investigation, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This paper is supported by Anhui Province University Excellent Top Talent Training Project (gxbjZD2022023), Anhui Province Key Laboratory of Machine Vision Detection and Perception (KLMVI-2024-HIT-15).

Data availability

No data was used for the research described in the article.

References

Bo, L., Zhang, Z., Liu, Y., Yang, S., Wang, Yanwen, Wang, Yiying, Zhang, X., 2024. Research on path planning method of solid backfilling and pushing mechanism

based on adaptive genetic particle swarm optimization. Mathematics 12, 479. https://doi.org/10.3390/math12030479.
Chaari, I., Koubaa, A., Bennaceur, H., Ammar, A., Alajlan, M., Youssef, H., 2017. Design and performance analysis of global path planning techniques for autonomous mobile robots in grid environments. Int. J. Adv. Rob. Syst. 14. https://doi.org/10.1177/1729881416663663.
Chen, T., Chen, S., Zhang, K., Qiu, G., Li, Q., Chen, X., 2022. A jump point search improved ant colony hybrid optimization algorithm for path planning of mobile robot. Int. J. Adv. Rob. Syst. 19, 172988062211279. https://doi.org/10.1177/17298806221127953.
Chen, W., Yang, J., Zhang, S., Wei, X., Liu, C., Zhou, X., Sun, L., Wang, F., Wang, A., 2025. Variable scale operational path planning for land levelling based on the improved ant colony optimization algorithm. Sci. Rep. 15. https://doi.org/10.1038/s41598-025-94008-y.
Cui, J., Wu, L., Huang, X., Xu, D., Liu, C., Xiao, W., 2024. Multi-strategy adaptable ant colony optimization algorithm and its application in robot path planning. Knowledge-based Syst. 288, 111459. https://doi.org/10.1016/j.knosys.2024.111459.
Cui, M., He, M., Chen, H., Liu, K., Hu, Y., Zheng, C., Wang, X., 2025. Path planning for mobile robot based on improved ant colony Q-learning algorithm. Int. J. Interact. Des. Manuf. https://doi.org/10.1007/s12008-025-02241-6.
Cui, Y., Ren, J., Zhang, Y., 2022. Path planning algorithm for unmanned surface vehicle based on optimized ant colony algorithm. IEEJ Transactions Elec Engng 17, 1027–1037. https://doi.org/10.1002/tee.23592.
Gao, Y., Wu, H., Wang, W., 2022. A hybrid ant colony optimization with fireworks algorithm to solve capacitated vehicle routing problem. Appl. Intell. 53, 7326–7342. https://doi.org/10.1007/s10489-022-03912-7.
Gui, Y., Tang, D., Zhu, H., Zhang, Y., Zhang, Z., 2023. Dynamic scheduling for flexible job shop using a deep reinforcement learning approach. Comput. Ind. Eng. 180, 109255. https://doi.org/10.1016/j.cie.2023.109255.
Guo, H., Qiu, Z., Gao, G., Wu, T., Chen, H., Wang, X., 2024. Safflower picking trajectory planning strategy based on an ant colony genetic fusion algorithm. Agriculture 14, 622. https://doi.org/10.3390/agriculture14040622.
Han, Z., Chen, M., Shao, S., Zhou, T., Wu, Q., 2023. Path planning of unmanned autonomous helicopter based on hybrid satisficing decision-enhanced swarm intelligence algorithm. IEEE Trans. Cogn. Dev. Syst. 15, 1371–1385. https://doi.org/10.1109/tcds.2022.3212062.
Hao, B., Du, H., Yan, Z., 2023. A path planning approach for unmanned surface vehicles based on dynamic and fast Q-learning. Ocean. Eng. 270, 113632. https://doi.org/10.1016/j.oceaneng.2023.113632.
Huang, L., Fu, Q., He, M., Jiang, D., Hao, Z., 2021. Detection algorithm of safety helmet wearing based on deep learning. Concurr. Comput. 33. https://doi.org/10.1002/cpe.6234.
Li, D., Wang, L., Cai, J., Ma, K., Tan, T., 2023. Research on terminal distance index-based multi-step ant colony optimization for Mobile robot path planning. IEEE Trans. Autom. Sci. Eng. 20, 2321–2337. https://doi.org/10.1109/tase.2022.3212428.
Li, M., Li, B., Qi, Z., Li, J., Wu, J., 2023. Optimized APF-ACO algorithm for ship collision avoidance and path planning. JMSE 11, 1177. https://doi.org/10.3390/jmse11061177.
Li, P., Wei, L., Wu, D., 2025. An intelligently enhanced ant colony optimization algorithm for global path planning of Mobile robots in engineering applications. Sensors 25, 1326. https://doi.org/10.3390/s25051326.
Li, S., Yin, G., Yan, Z., Liu, Z., 2023. Intelligent agents path planning in wireless sensor networks based on Vor-PSO algorithm. IJBIC 1, 1. https://doi.org/10.1504/ijbic.2023.10054855.
Liang, C., Pan, K., Zhao, M., Lu, M., 2023. Multi-node path planning of electric tractor based on improved whale optimization algorithm and ant colony algorithm. Agriculture 13, 586. https://doi.org/10.3390/agriculture13030586.
Lin, S., Liu, A., Wang, J., Kong, X., 2023. An intelligence-based hybrid PSO-SA for mobile robot path planning in warehouse. J. Comput. Sci. 67, 101938. https://doi.org/10.1016/j.jocs.2022.101938.
Liu, Z., Liu, J., 2023. Improved ant colony algorithm for path planning of mobile robots based on compound prediction mechanism. IFS 44, 2147–2162. https://doi.org/10.3233/jifs-222211.
Miao, C., Chen, G., Yan, C., Wu, Y., 2021. Path planning optimization of indoor mobile robot based on adaptive ant colony algorithm. Comput. Ind. Eng. 156, 107230. https://doi.org/10.1016/j.cie.2021.107230.
Ni, Y., Zhuo, Q., Li, N., Yu, K., He, M., Gao, X., 2023. Characteristics and optimization strategies of A* algorithm and ant colony optimization in global path planning algorithm. Int. J. Pattern Recogn. Artif. Intell. 37, 2351006. https://doi.org/10.1142/s0218001423510060.
Ni, Z., Cai, S., Ni, C., 2023. Research on energy-saving strategy of wireless sensor network based on improved ant colony algorithm. Sensor. Mater. 35, 1835. https://doi.org/10.18494/sam4310.
Ning, Y., Zhang, F., Jin, B., Wang, M., 2023. Three-dimensional path planning for a novel sediment sampler in ocean environment based on an improved mutation operator genetic algorithm. Ocean. Eng. 289, 116142. https://doi.org/10.1016/j.oceaneng.2023.116142.
Pan, G., Xiang, Y., Wang, X., Yu, Z., Zhou, X., 2022. Research on path planning algorithm of Mobile robot based on reinforcement learning. Soft Comput. 26, 8961–8970. https://doi.org/10.1007/s00500-022-07293-4.
Rao, J., Xiang, C., Xi, J., Chen, J., Lei, J., Giernacki, W., Liu, M., 2023. Path planning for dual UAVs cooperative suspension transport based on artificial potential field-A* algorithm. Knowledge-based Syst. 277, 110797. https://doi.org/10.1016/j.knosys.2023.110797.
Shi, K., Huang, L., Jiang, D., Sun, Y., Tong, X., Xie, Y., Fang, Z., 2022. Path planning optimization of intelligent vehicle based on improved genetic and ant colony hybrid algorithm. Front. Bioeng. Biotechnol. 10, 905483. https://doi.org/10.3389/fbioe.2022.905983.
Sun, H., Zhu, K., Zhang, W., Ke, Z., Hu, H., Wu, K., Zhang, T., 2025. Emergency path planning based on improved ant colony algorithm. J. Build. Eng. 111725–111725. https://doi.org/10.1016/j.jobe.2024.111725.
Sun, R., Yang, T., 2023. Hybrid parameter-based PSO flexible needle percutaneous puncture path planning. J. Supercomput. 80, 5408–5427. https://doi.org/10.1007/s11227-023-05661-x.
Tan, A., Wang, C., Wang, Y., Dong, C., 2024. Electric vehicle charging route planning for shortest travel time based on improved ant colony optimization. Sensors 25, 176. https://doi.org/10.3390/s25010176.
Tan, Y., Ouyang, J., Zhang, Z., Lao, Y., Wen, P., 2022. Path planning for spot welding robots based on improved ant colony algorithm. Robotica 41, 926–938. https://doi.org/10.1017/s026357472200114x.
Tian, J., Cheng, W., Sun, Y., Li, G., Jiang, D., Jiang, G., Tao, B., Zhao, H., Chen, D., 2020. Gesture recognition based on multilevel multimodal feature fusion. IFS 38, 2539–2550. https://doi.org/10.3233/jifs-179541.
Wang, L., 2020. Path planning for unmanned wheeled robot based on improved ant colony optimization. Meas. Control 53, 1014–1021. https://doi.org/10.1177/0020294020909129.
Wang, Y., Han, Z., 2021. Ant colony optimization for traveling salesman problem based on parameters optimization. Appl. Soft Comput. 107, 107439. https://doi.org/10.1016/j.asoc.2021.107439.
Wang, Y., Lu, C., Wu, P., Zhang, X., 2024. Path planning for unmanned surface vehicle based on improved Q-Learning algorithm. Ocean. Eng. 292, 116510. https://doi.org/10.1016/j.oceaneng.2023.116510.
Wu, J., Ma, X., Peng, T., Wang, H., 2021. An improved Timed Elastic Band (TEB) algorithm of autonomous ground vehicle (AGV) in complex environment. Sensors 21, 8312. https://doi.org/10.3390/s21248312.
Wu, L., Huang, X., Cui, J., Liu, C., Xiao, W., 2023. Modified adaptive ant colony optimization algorithm and its application for solving path planning of mobile robot. Expert Syst. Appl. 215, 119410. https://doi.org/10.1016/j.eswa.2022.119410.
Wu, Y., Wu, S., Hu, X., 2021. Cooperative path planning of UAVs & UGVs for a persistent surveillance task in urban environments. IEEE Internet Things J. 8, 4906–4919. https://doi.org/10.1109/jiot.2020.3030240.
Xiang, S.-Y., Zhou, H.-F., Huo, J.-W., Zhang, H., Gu, C.-F., 2025. A dynamic nuclear radiation environment path planning method based on an improved ant colony algorithm. Nucl. Technol. 1–20. https://doi.org/10.1080/00295450.2025.2457249.
Xu, S., Bi, W., Zhang, A., Wang, Y., 2023. A deep reinforcement learning approach incorporating genetic algorithm for missile path planning. Int. J. Mach. Learn. & Cyber 15, 1795–1814. https://doi.org/10.1007/s13042-023-01998-0.
Yang, H., Qi, J., Miao, Y., Sun, H., Li, J., 2019. A new robot navigation algorithm based on a double-layer ant algorithm and trajectory optimization. IEEE Trans. Ind. Electron. 66, 8557–8566. https://doi.org/10.1109/tie.2018.2886798.
Yang, Z., Fang, L., Zhang, X., Zuo, H., 2021. Controlling a scattered field output of light passing through turbid medium using an improved ant colony optimization algorithm. Opt Laser. Eng. 144, 106646. https://doi.org/10.1016/j.optlaseng.2021.106646.
Zhang, S., Pu, J., Si, Y., Sun, L., 2021. Path planning for mobile robot using an enhanced ant colony optimization and path geometric optimization. Int. J. Adv. Rob. Syst. 18, 172988142110192. https://doi.org/10.1177/17298814211019222.
Zhang, Z.-H., Wang, J.-S., Chen, L., 2024. An edge detection method of colony image based on mediocrity ant colony algorithm. IFS 46, 2665–2691. https://doi.org/10.3233/jifs-233769.
Zhao, L., Li, F., Sun, D., Zhao, Z., 2024. An improved ant colony algorithm based on Q-Learning for route planning of autonomous vehicle. INT J COMPUT COMMUN 19. https://doi.org/10.15837/ijccc.2024.3.5382.
Zhou, Q., Lian, Y., Wu, J., Zhu, M., Wang, H., Cao, J., 2024. An optimized Q-Learning algorithm for mobile robot local path planning. Knowledge-based Syst. 286, 111400. https://doi.org/10.1016/j.knosys.2024.111400.